

JAVASCRIPT CURRICULUM • MODULE 05

The Complete JavaScript String & Text Processing Encyclopedia

From UTF-16 Code Units & Regular Expressions to High-Performance Algorithms & Production Architecture

Beginner → Senior Masterclass

79 Verified Algorithms

4 Real-World Projects

ECMAScript & V8 Standards

Module 05 — Complete JavaScript Strings

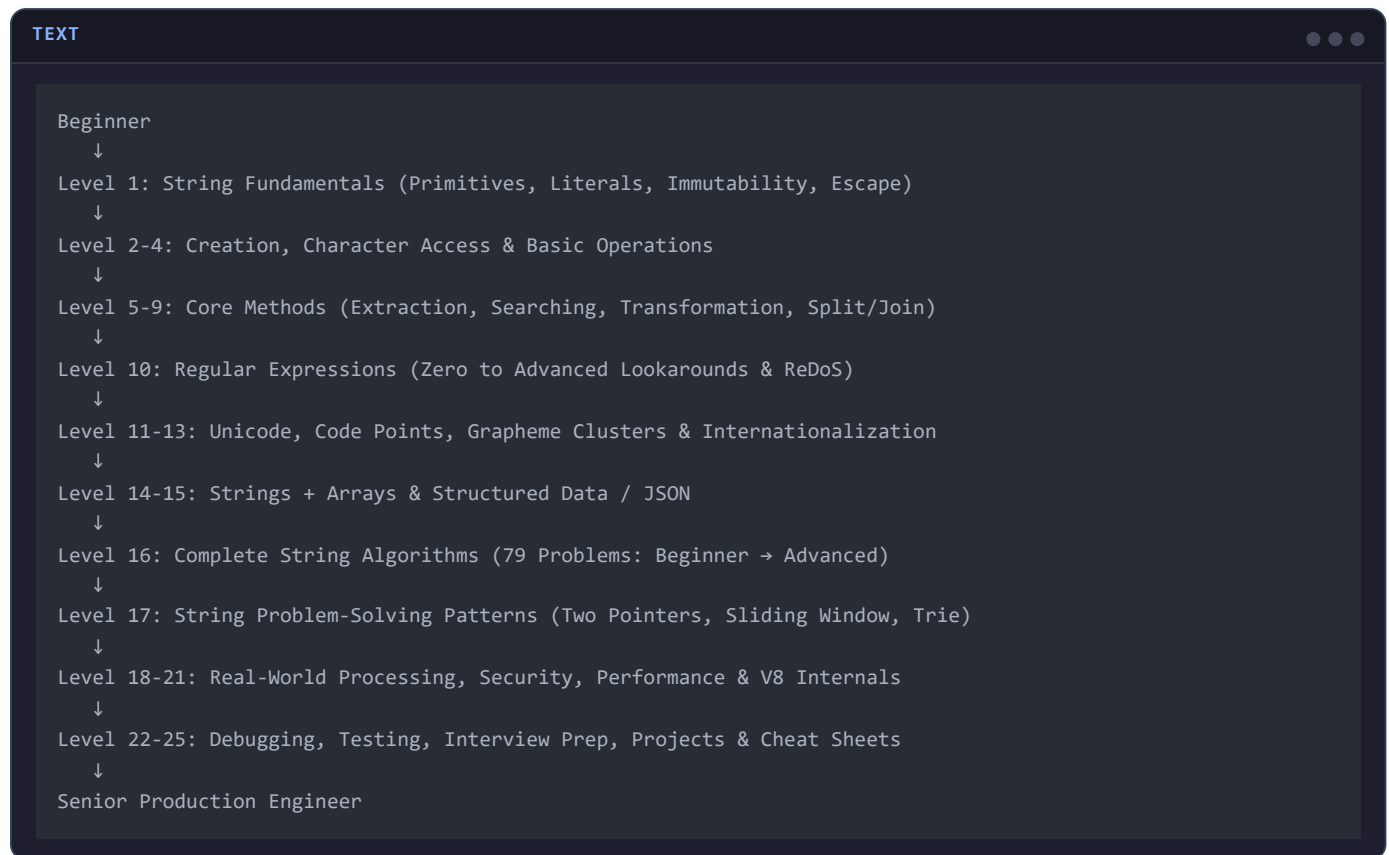
Beginner → Advanced → Senior

Core Invariant: In JavaScript, strings are **immutable primitives**. Every character in standard text is represented by 16-bit code units in UTF-16 encoding. Once allocated in memory, a string can **never be modified in place**. Every single string method returns a **brand new string**.

00. How to Use This Module & Roadmap

Welcome to the comprehensive JavaScript String Learning Guide. This module is engineered to take you from foundational understanding to production-level systems engineering.

Learning Roadmap



Visual Legend Used Throughout This Guide

- **Concept:** Fundamental theory, mental model, or specification rule.
- **Important:** Core principle to commit to memory.
- **Common Mistake:** Bugs that trip up developers in production.
- **Deep Dive:** Low-level engine behavior, UTF-16, or memory architecture.
- **Real World:** Production backend, cloud storage, or API scenarios.
- **Practice:** Hands-on mini exercises.
- **Challenge:** Complex architectural coding problem.
- **Interview:** Questions asked in senior engineering interviews.
- **Performance:** Algorithmic complexity, V8 optimization, memory overhead.
- **Security:** Defensive coding, injection prevention, input sanitization.

01. String Fundamentals

1. What is a String?

A **string** is an ordered sequence of zero or more 16-bit characters used to represent text. In JavaScript, strings are one of the 7 primitive data types.

2. Primitive Strings vs String Literals

A primitive string is stored by value. You write primitive strings using **string literals**:

- **Single Quotes:** `'hello'`
- **Double Quotes:** `"hello"`

- **Template Literals:** ``hello``

All three create identical primitive strings in memory:

```
JAVASCRIPT

const s1 = 'Single quote string';
const s2 = "Double quote string";
const s3 = `Template literal string`;

console.log(s1 === "Single quote string"); // true
console.log(s2 === 'Double quote string'); // true
```

3. Empty Strings

An empty string `""` is a string with a length of `0`.

```
JAVASCRIPT

const empty = "";
console.log(typeof empty); // "string"
console.log(empty.length); // 0
console.log(Boolean(empty)); // false (Empty string is falsy!)
```

4. typeof Operator with Strings

```
JAVASCRIPT

console.log(typeof "cloud"); // "string"
console.log(typeof ""); // "string"
console.log(typeof String(123)); // "string"
```

5. String Length Property (.length)

The `.length` property returns the number of **16-bit code units** in the string:

```
JAVASCRIPT

const filename = "report.pdf";
console.log(filename.length); // 10
```

(⚠ Note: For characters outside the basic multilingual plane, like emojis, length reflects code units, not perceived characters. We explore this deeply in Level 11).

6. String Immutability

Once a string is created, its content **can never be altered**:

JAVASCRIPT

```
let word = "cat";  
word[0] = "b"; // Trying to change 'c' to 'b'  
console.log(word); // "cat" (UNTOUCHED!)
```

To change text, you must create a new string and assign it:

JAVASCRIPT

```
word = "b" + word.slice(1);  
console.log(word); // "bat"
```

7. Escape Characters

Backslash (`\`) introduces escape sequences:

- `\n`: Newline (line break)
- `\t`: Horizontal tab
- `\r`: Carriage return
- `\\`: Literal backslash
- `\'`: Single quote
- `\"`: Double quote
- `\``: Backtick

JAVASCRIPT

```
const pathWindows = "C:\\Users\\ayush\\documents";  
const multiLine = "Line 1\\nLine 2";  
const quote = "He said, \\\"Deploy now!\\\"";
```

8. Unicode Escape Syntax

- `\uXXXX`: 4-digit hexadecimal code unit (e.g., `\u0041` is `'A'`).
- `\u{XXXXX}` (ES6): Code point escape for characters requiring up to 6 hex digits:

JAVASCRIPT

```
console.log("\u0041"); // "A"  
console.log("\u{1F4C1}"); // "📁" (Folder emoji)
```

9. String Conversion: `String()` vs `.toString()`

- `String(val)`: Converts any value (including `null` and `undefined`) safely to a string without throwing errors:

JAVASCRIPT

```
console.log(String(100));    // "100"
console.log(String(null));   // "null"
console.log(String(undefined)); // "undefined"
console.log(String(true));   // "true"
```

- `val.toString()` : Method on objects and primitives (via wrapper). Throws `TypeError` if called on `null` or `undefined` :

JAVASCRIPT

```
const num = 255;
console.log(num.toString());    // "255"
console.log(num.toString(16));  // "ff" (Radix 16: hex representation!)

const empty = null;
// empty.toString(); // ❌ THROWS TypeError: Cannot read properties of null
```

10. String Coercion Rules

When the `+` operator encounters a string operand, it converts the other operand to a string:

JAVASCRIPT

```
console.log("File " + 1);        // "File 1" (Number 1 becomes "1")
console.log("Is active: " + true); // "Is active: true"
console.log("Value: " + null);    // "Value: null"
console.log("Value: " + undefined); // "Value: undefined"
console.log("Items: " + [1, 2]);  // "Items: 1,2" (Array coerced via [].toString())
```

02. String Creation & Objects

Primitive vs String Object (new String())

JAVASCRIPT

```
const prim = "hello";           // Primitive string
const obj = new String("hello"); // String wrapper object instance on the Heap
```

Why new String() Should Normally Be Avoided

1. Type Inequality:

JAVASCRIPT

```
typeof prim; // "string"
typeof obj;   // "object"
prim === obj; // false! (Primitive vs Object reference)
```

2. Truthiness Hazard:

JAVASCRIPT

```
const emptyObj = new String("");
if (emptyObj) {
  // This BLOCK RUNS! ALL objects in JavaScript are TRUTHY!
  console.log("Empty object string is truthy!");
}
```

Autoboxing (How Primitives Access Methods)

Primitives have no methods. When you call `"hello".toUpperCase()` :

1. The engine conceptually wraps the primitive in a temporary wrapper object (`Object("hello")`).
2. Calls the method `String.prototype.toUpperCase` from the prototype.
3. Returns the resulting new primitive string.
4. Discards the temporary wrapper. (⚡ *Note: Modern JavaScript engines like V8 optimize this via inline caching, avoiding actual physical heap allocation for simple method reads.*)

03. Immutability & Coercion

The Immutability Invariant

Strings cannot be altered in place. Every transformation allocates a new string:

JAVASCRIPT

```
let str = "cloud";
str.toUpperCase(); // Returns "CLOUD", but does NOT alter 'str'
console.log(str); // "cloud"

str = str.toUpperCase(); // Explicit reassignment saves the new string
console.log(str); // "CLOUD"
```

Coercion Mechanics in Comparison

- **Strict Equality** (`===`): No coercion. Types must match:

JAVASCRIPT

```
"42" === 42; // false
```

- **Loose Equality** (`==`): Coerces operands before comparison. A major source of production bugs:

JAVASCRIPT

```
"42" == 42; // true
"" == 0; // true
"\n" == 0; // true
```

💡 **Invariant:** Always use `===` for string comparisons.

04. Character Access: `[]` vs `.charAt()` vs `.at()`

In JavaScript, think of a string like an apartment building or a row of numbered lockers. Every single character lives in its own numbered room, starting at **Room 0**.

TEXT					
Locker Number:	0	1	2	3	4
Character:	H	e	l	l	o
Reverse Index:	-5	-4	-3	-2	-1

JavaScript gives you **3 different ways** to knock on a door and ask for the character inside:

1. `str[index]` (The Casual Bracket)
2. `str.charAt(index)` (The Polite Antique Method)
3. `str.at(index)` (The Smart Modern Champion 🏆 — ES2022)

What is it in Plain English?

1. `str[index]` — The Casual Bracket

Just like accessing an array item: `str[0]` gives you the first character.

- **If the room doesn't exist** (out of bounds, like `str[99]`): Gives you `undefined`.
- **Negative numbers?** Does NOT work! `str[-1]` gives `undefined`.

2. `str.charAt(index)` — The Antique Method

The original method from 1995:

- **If the room doesn't exist:** Returns an empty string `""` (never `undefined`).
- **Negative numbers?** Treats any negative number as `< 0` and returns an empty string `""`.

3. `str.at(index)` — The Modern Champion (ES2022) 🏆

The cleanest, most intuitive modern method:

- **Supports Negative Numbers (The Rear Bumper!):**
 - `str.at(-1)` grabs the **very last character**!
 - `str.at(-2)` grabs the **second to last character**!
 - You never have to write the clumsy `str[str.length - 1]` ever again.
- **If the room doesn't exist:** Returns `undefined`.

Comparison Matrix: Which One Should You Use?

Expression	str[index]	str.charAt(index)	str.at(index)
str[0]	First character	First character	First character
Out of bounds (100)	undefined	"" (Empty string)	undefined
Negative index (-1)	undefined (Fails!)	"" (Fails!)	Last character! 🎯
Negative index (-2)	undefined (Fails!)	"" (Fails!)	Second to last! 🎯
Boolean index (true)	str[true] -> undef	str.charAt(true)->[1]	str.at(true) -> [1]
Recommendation	Great for 0+ indices	Legacy code only	Best for modern code

Practical Code Examples

```

JAVASCRIPT

const filename = "invoice_2026.pdf";

// 1. Grabbing first character:
console.log(filename[0]);           // "i"
console.log(filename.charAt(0));    // "i"
console.log(filename.at(0));        // "i"

// 2. Grabbing Last character (Notice how much cleaner .at(-1) is!):
console.log(filename[filename.length - 1]); // "f" (Old, verbose way)
console.log(filename.at(-1));              // "f" (Clean, modern ES2022!)
console.log(filename.at(-2));              // "d"
console.log(filename.at(-3));              // "p"

// 3. Out of bounds safety:
console.log(filename[50]);                // undefined
console.log(filename.charAt(50));          // "" (Empty string)
console.log(filename.at(50));              // undefined

```

05. String Iteration

Why "hello".map() Does NOT Work

Strings are primitives and do not inherit from `Array.prototype`. Calling `"hello".map()` throws: `TypeError: "hello".map is not a function`.

Converting Strings to Arrays for Processing Pipeline:

TEXT

```
String
↓
split('') / [...str] / Array.from(str)
↓
Array
↓
map() / filter() / reduce()
↓
join('')
↓
String
```

Methods of Iterating Strings:

1. Traditional for loop (Code unit by code unit)

JAVASCRIPT

```
const str = "Drive";
for (let i = 0; i < str.length; i++) {
  console.log(str[i]);
}
```

2. for...of loop (Code point aware)

JAVASCRIPT

```
// Respects surrogate pairs (emojis):
for (const char of "Hi 📁") {
  console.log(char); // "H", "i", " ", "📁"
}
```

3. Spread Syntax ([...str])

JAVASCRIPT

```
const chars = [..."code"]; // ["c", "o", "d", "e"]
```

4. Array.from(str)

JAVASCRIPT

```
const upperChars = Array.from("file", c => c.toUpperCase());
console.log(upperChars); // ["F", "I", "L", "E"]
```

06. Core String Properties & Concatenation

The .length Property

JAVASCRIPT

```
console.log("cloud".length); // 5
console.log("").length;      // 0
```

Concatenation Options

1. Plus operator (+ / +=):

JAVASCRIPT

```
let full = "hello" + " " + "world";
```

2. Template Literals:

JAVASCRIPT

```
const greeting = `${full} from Cloud Storage`;
```

3. .concat() Method:

JAVASCRIPT

```
const c = "a".concat("b", "c"); // "abc"
```

💡 **Best Practice:** Use Template Literals for formatting and dynamic variables. Use `+` for simple joining. Avoid `.concat()` as it is slower and more verbose.

07. Extraction: slice() vs substring() vs substr()

Extracting text means taking a smaller slice out of a bigger string.

💡 **Core Invariant:** Remember, JavaScript strings are **immutable** (locked down). Extraction methods **never chop or change the original string**. They leave the original untouched and hand you a **brand new string** containing the extracted slice.

TEXT

Forward Index:	0	1	2	3	4	5	6	7	8	9											
Characters:		J		A		V		A		S		C		R		I		P		T	
Reverse Index:	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1											

JavaScript has three extraction tools. Think of them like this:

- `slice()` = **The Clean Ruler & Cake Slicer** 🍰: Modern, predictable, supports negative numbers. **(Always use this!)**
- `substring()` = **The Forgiving Safety Net** 🛡️: Legacy tool that auto-swaps your numbers if you put them backwards, and ignores negatives.
- `substr()` = **The Antique Tool** 🕯️: Deprecated old tool. Do not use.

Comparison Matrix: At a Glance

Feature	<code>str.slice(start, end)</code>	<code>str.substring(start, end)</code>	<code>str.substr(start, length)</code>
Meaning of 2nd argument	End index (exclusive)	End index (exclusive)	Character count (length)
Negative start index	Counts backward from the end	Treated as 0	Counts backward from the end
Negative end index	Counts backward from the end	Treated as 0	N/A (length cannot be negative)
When start > end	Returns empty string ""	Swaps start and end arguments	Extracts 'length' character count
NaN or undefined start	Treated as 0	Treated as 0	Treated as 0
Recommendation	Modern Standard (100% Best)	Legacy Only	⚠️ Deprecated (Do not use)

1. `str.slice(startIndex[, endIndex])` — The Cake Slicer 🍰 (Modern Standard)

What is it in Plain English?

Imagine placing two cut marks on a cake or ribbon. `slice()` cuts between those two marks and hands you the slice.

Formal Syntax

```
JAVASCRIPT
str.slice(startIndex)
str.slice(startIndex, endIndex)
```

The 3 Golden Rules of `slice()`:

1. `endIndex` is **EXCLUSIVE**: The cut stops right **before** `endIndex`. The character *at* `endIndex` is **never** included!

```
JAVASCRIPT
// Indices 0, 1, 2, 3 (stops before index 4):
"JavaScript".slice(0, 4); // "Java"
```

2. Negative numbers count from the back bumper:

JAVASCRIPT

```
// Give me the last 3 characters:  
"archive.tar.gz".slice(-2); // "gz"  
// Slice starting from index 0, stopping 3 characters before the end:  
"hello world".slice(0, -6); // "hello"
```

3. If `start >= end`, it returns an empty string `""`: No weird guessing or swapping. If your start mark is past your end mark, there is no cake between them:

JAVASCRIPT

```
"Cloud".slice(4, 1); // ""
```

2. `str.substring(startIndex[, endIndex])` — The Forgiving Safety Net 🛡️

What is it in Plain English?

`substring` was designed back in 1995 to be forgiving to beginners who made mistakes with their numbers:

1. **It Auto-Swaps Backwards Numbers:** If you accidentally say "cut from index 10 to index 4", `substring` says: "You probably meant 4 to 10!" and automatically swaps them for you:

JAVASCRIPT

```
"JavaScript".substring(10, 4); // Automatically treated as substring(4, 10) -> "Script"
```

2. **It Treats All Negative Numbers as Zero** `0`: You cannot use negative numbers with `substring`:

JAVASCRIPT

```
"hello".substring(-3); // Automatically treated as substring(0) -> "hello"
```

💡 **Senior Developer Tip:** While auto-swapping sounds friendly, in production software it can silently hide math bugs where your index calculation was completely wrong. **Stick to** `slice()`.

3. `str.substr(start[, length])` — ⚠️ Deprecated (Do Not Use)

Why is it Deprecated?

Instead of an end position, the second argument was a *count* (how many characters to grab). It has been officially deprecated in ECMAScript (Annex B). Modern codebases should never write `substr()`. Always use `slice()`.

5 Graded Real-World Production Examples

Example 1: Extracting File Extension & Base Name

JAVASCRIPT

```
function parseFilename(filepath) {
  const normalized = filepath.replaceAll("\\\\", "/");
  const filename = normalized.slice(normalized.lastIndexOf("/") + 1);
  const dotIndex = filename.lastIndexOf(".");

  if (dotIndex <= 0) return { baseName: filename, extension: "" };

  return {
    baseName: filename.slice(0, dotIndex),
    extension: filename.slice(dotIndex + 1)
  };
}

console.log(parseFilename("/var/data/monthly_report.2026.pdf"));
// { baseName: "monthly_report.2026", extension: "pdf" }
```

Example 2: Truncating Text with a Middle Ellipsis ("...")

JAVASCRIPT

```
function truncateMiddle(str, maxLength = 20) {
  if (str.length <= maxLength) return str;
  const charsToShow = maxLength - 3;
  const frontChars = Math.ceil(charsToShow / 2);
  const backChars = Math.floor(charsToShow / 2);

  // Front slice + dots + back slice (using negative index!):
  return str.slice(0, frontChars) + "..." + str.slice(-backChars);
}

console.log(truncateMiddle("user_upload_production_cluster_key_0982.pem", 24));
// "user_upload_...y_0982.pem"
```

Example 3: Masking Sensitive Payment Cards

JAVASCRIPT

```
function maskCard(cardNumber) {
  const digits = cardNumber.replace(/\D/g, "");
  if (digits.length < 4) return digits;

  const lastFour = digits.slice(-4);
  return "*".repeat(digits.length - 4) + lastFour;
}

console.log(maskCard("4532-8921-7734-1098")); // "*****1098"
```

Example 4: Breadcrumbs from URL Path

JAVASCRIPT

```
function getBreadcrumbs(urlPath) {
  const clean = urlPath.slice(
    urlPath.startsWith("/") ? 1 : 0,
    urlPath.endsWith("/") ? -1 : undefined
  );
  return clean.split("/");
}

console.log(getBreadcrumbs("/api/v2/cloud/buckets/billing/"));
// ["api", "v2", "cloud", "buckets", "billing"]
```

Example 5: 🚩 The Emoji & Surrogate Pair Trap with slice()

JAVASCRIPT

```
// Emojis like 🏠 take TWO 16-bit units in memory.
const text = "Hi 🏠 team";

// Slicing right through the middle of an emoji corrupts it!
console.log(text.slice(0, 4)); // "Hi 🏠" (Broken half-emoji!)

// ✅ Safe Modern Solution: Array.from or spread preserves the full emoji:
function safeSlice(str, start, end) {
  return Array.from(str).slice(start, end).join("");
}

console.log(safeSlice(text, 0, 4)); // "Hi 🏠"
```

08. Searching Strings

Searching and locating text patterns is fundamental to validation, routing, text parsing, and string manipulation. JavaScript provides 8 distinct search methods, each optimized for different algorithmic requirements and return types.

Decision Table: "Which Search Method Should I Use?"

TEXT

Question / Goal	Recommended Method	Return Type
Need boolean: does substring exist?	<code>.includes(searchStr)</code>	boolean
Need boolean: does it start with prefix?	<code>.startsWith(prefix)</code>	boolean
Need boolean: does it end with suffix?	<code>.endsWith(suffix)</code>	boolean
Need position of first occurrence?	<code>.indexOf(searchStr)</code>	number (index or -1)
Need position of last occurrence?	<code>.lastIndexOf(searchStr)</code>	number (index or -1)
Need position matching Regular Expression?	<code>.search(regex)</code>	number (index or -1)
Need match details (single or array)?	<code>.match(regex)</code>	Array null
Need all matches + capture groups (iterator)?	<code>.matchAll(regexWithGlobalG)</code>	RegExpStringIterator

1. `str.indexOf(searchValue[, fromIndex])`

What is it in Plain English?

`indexOf` scans your string from left to right to find **where** a substring first appears.

- If found, it hands you the 0-based index (e.g. `0`, `1`, `4`).
- If not found anywhere, it hands you `-1`.

One Core Mental Model 🧠

Whenever you write:

JAVASCRIPT

```
str.indexOf("a", 2);
```

Think of it as asking JavaScript a simple human question:

"Starting from index 2, where is the very first 'a'?"

And JavaScript only gives you two kinds of answers:

- **Found** → The exact index number where it starts
- **Not found** → `-1`

The 4 Golden Rules to Remember

1. **Finds the first occurrence:** It stops searching the moment it hits the first match.
2. **Returns the index:** A 0-based position number.
3. **Not found is always `-1`:** Never `null`, `undefined`, or `false`.
4. `fromIndex` **is where searching begins:** It tells the engine where to start looking. If the character at `fromIndex` matches, it immediately returns that index!

⚠️ BUT There's One Weird Exception: The Empty String ""

This is the part that usually catches developers off guard:

JAVASCRIPT

```
str.indexOf("");
```

You might naturally think: *"I'm searching for nothing, so shouldn't it return -1?"*

No. JavaScript specifies that an empty string `""` is considered to exist between every character, and is found **immediately at the starting position**:

- `str.indexOf("")` returns `0` (because the default `fromIndex` is `0`).
- `str.indexOf("", 4)` returns `4` (because you told it to start at index `4`).
- `str.indexOf("", 100)` returns `10` (if `str.length` is `10`, because it never goes beyond the string's length).

💡 **The Simple Rule for `""`:** Searching for `""` returns the starting position (`fromIndex`), but never beyond `str.length`. (This is a specification edge-case you will rarely need in everyday code, but it's essential for interviews!)

🔥 Mini Practice Task

Don't run these in a console—try to predict the output in your head using the mental model:

TEXT

Index:	0	1	2	3	4	5
Letter:	b	a	n	a	n	a

JAVASCRIPT

```
const str = "banana";

str.indexOf("a");    // ?
str.indexOf("a", 2); // ?
str.indexOf("a", 3); // ?
str.indexOf("z");    // ?
```

Answers & Explanations in Order:

- `str.indexOf("a")` → `1`: Starts searching at index 0. The first `'a'` is at index 1.
- `str.indexOf("a", 2)` → `3`: Starts searching at index 2 (`'n'`). Moving forward, the first `'a'` appears at index 3.
- `str.indexOf("a", 3)` → `3`: Starts searching right at index 3. Because index 3 IS `'a'` , it immediately matches at index 3! (`fromIndex` is inclusive).
- `str.indexOf("z")` → `-1`: The letter `'z'` does not exist in `"banana"` , so it returns `-1` .

Formal Syntax & Signatures

JAVASCRIPT

```
str.indexOf(searchValue)
str.indexOf(searchValue, fromIndex)
```

Parameter Details

- `searchValue` (*string*): What you want to find. Non-strings are coerced to string.
- `fromIndex` (*integer, optional, default = `0`*): The starting index.
 - Negative numbers (`-5`) are treated as `0` (scans from start).
 - Values `>= str.length` return `-1` (unless searching for `""`).

Real-World Development Usage & Code Examples

JAVASCRIPT

```
const filename = "user.profile.avatar.png";

// 1. Finding delimiter boundaries:
const firstDot = filename.indexOf(".");
console.log(firstDot); // 4 (Boundary between "user" and "profile")

// 2. Finding all occurrences in a loop:
function findAllPositions(text, char) {
  const matches = [];
  let index = text.indexOf(char);
  while (index !== -1) {
    matches.push(index);
    index = text.indexOf(char, index + 1); // Advance 1 step past match
  }
  return matches;
}
console.log(findAllPositions("banana", "a")); // [1, 3, 5]

// 3. Case-sensitive trap:
console.log("JavaScript".indexOf("script")); // -1 (Capital 'S' required!)
```

2. str.lastIndexOf(searchValue[, fromIndex]) — The Rearview Mirror 🚗

What is it in Plain English?

While `indexOf` scans from **left to right** (forward), `lastIndexOf` scans from **right to left** (backward). It looks for the **very last time** a word or character appears in your string.

One Core Mental Model 🗨️

Whenever you write:

JAVASCRIPT

```
str.lastIndexOf("cat");
```

Think of it as asking:

"Starting from the very end of the string and looking backwards, where is the first 'cat' I bump into?"

- **Found** → Returns that 0-based index.
- **Not found** → Returns `-1`.

Real-World Dev Use Cases

- Finding file extensions: In filenames like `"bundle.min.v2.js"`, you want the **last dot** `.`, not the first one!
- Finding folder paths: Finding the last slash `/` or `\\` to get the file name from a long path.

Practical Code Examples

JAVASCRIPT

```
const path = "src/modules/billing/invoice.service.ts";

// 1. Finding the last slash to extract filename:
const lastSlash = path.lastIndexOf("/");
const filename = path.slice(lastSlash + 1);
console.log(filename); // "invoice.service.ts"

// 2. Finding the last dot to get the real file extension:
const lastDot = filename.lastIndexOf(".");
console.log(filename.slice(lastDot)); // ".ts"

// 3. Searching backwards with fromIndex:
const sentence = "the cat and the other cat";
console.log(sentence.lastIndexOf("cat")); // 22 (Final "cat")
console.log(sentence.lastIndexOf("cat", 15)); // 4 (Looks backward from index 15!)
```

3. `str.includes(searchString[, position])` — The Yes/No Lie Detector 🕵️

What is it in Plain English?

`includes` does not care *where* something is. It only gives you a clean boolean: `true` or `false`.

One Core Mental Model 🗨️

Stop writing the clumsy legacy check:

JAVASCRIPT

```
// ❌ Old legacy way (Harder to read):
if (email.indexOf("@") !== -1) { ... }

// ✅ Clean modern way (Reads like plain English!):
if (email.includes("@")) { ... }
```

The 3 Rules to Remember:

1. **Case-Sensitive:** `"Hello".includes("hello")` is `false`.
2. **Returns Boolean:** Only `true` or `false`.
3. **Optional `position`:** Tells it where to start checking.

Practical Code Examples

JAVASCRIPT

```
const roles = "ROLE_ADMIN,ROLE_EDITOR,ROLE_VIEWER";

if (roles.includes("ROLE_ADMIN")) {
  console.log("Welcome, Administrator!");
}

// Case-Insensitive Helper Utility:
function hasWord(text, word) {
  return text.toLowerCase().includes(word.toLowerCase());
}

console.log(hasWord("Cloud Storage", "storage")); // true
```

4. `str.startsWith(searchString[, position])` — The Security Guard at the Door 🚪

What is it in Plain English?

Checks whether a string begins with a specific prefix. Returns `true` or `false`.

Real-World Dev Use Cases

- Validating secure URLs: `url.startsWith("https://")`.
- API route routing: `req.path.startsWith("/api/v1")`.
- Detecting markdown elements: `line.startsWith("# ")`.
- CLI commands: `input.startsWith("--")`.

Practical Code Examples

JAVASCRIPT

```
const url = "https://cloud.google.com";

console.log(url.startsWith("https://")); // true
console.log(url.startsWith("http://")); // false

// Checking at a specific position offset:
console.log(url.startsWith("cloud", 8)); // true (Checks index 8!)
```

5. `str.endsWith(searchString[, endPosition])` — The Suffix Tag Inspector 🏷️

What is it in Plain English?

Checks whether a string finishes with a specific suffix. Returns `true` or `false`.

Real-World Dev Use Cases

- Validating uploaded files: `file.endsWith(".pdf") || file.endsWith(".png")`.
- Checking website domains: `email.endsWith("@company.com")`.

Practical Code Examples

JAVASCRIPT

```
function isImage(filename) {
  const lower = filename.toLowerCase();
  return lower.endsWith(".png") || lower.endsWith(".jpg") || lower.endsWith(".webp");
}

console.log(isImage("photo.PNG")); // true
console.log(isImage("notes.txt")); // false

// Ignoring query parameters using endPosition:
const url = "https://site.com/styles.css?v=2";
const questionMark = url.indexOf("?");
console.log(url.endsWith(".css", questionMark)); // true!
```

6. str.search(regexp) — The Pattern Radar 📡

What is it in Plain English?

`search` is like `indexOf`, but with **superpowers**: instead of searching for an exact literal word, it searches for a **Regular Expression pattern** (like "find the first number" or "find the first space").

- Returns the **0-based index** of the first match.
- Returns `-1` if no match is found.

Invariant to Remember

⚠️ **Ignores the `/g` flag entirely!** It always stops and returns the index of the **first** match.

Practical Code Examples

JAVASCRIPT

```
const log = "Timestamp: 2026-09-18 Error occurred";

// Find where the first number digit appears:
const firstDigitPos = log.search(/\d/);
console.log(firstDigitPos); // 11 (Where the '2' of 2026 starts)

// Find where the word "Error" or "Warning" appears:
const issuePos = log.search(/Error|Warning/);
console.log(issuePos); // 22
```

7. str.match(regexp) — The Treasure Chest / Collector 🏝️

What is it in Plain English?

`match` scans your string with a Regular Expression and collects matches into an Array.

The Big Split: How the `/g` Flag Changes Everything

1. **WITHOUT `/g` (Single Match + Capture Groups Treasure Chest)**: Catches the first match AND unpacks all parenthesized capture groups `()`:
 - `[0]`: Full matched string.

- `[1]`: First capture group.
- `.index`: Where the match was found.

2. WITH `/g` (**Vacuum Cleaner Mode**): Vaccums up **every single match** across the string into a simple array. (*Individual capture groups are omitted*).

💡 **Defensive Senior Trick**: If nothing matches, `match()` returns `null` (NOT an empty array!). Always do `str.match(...) || []` so your code never crashes with `TypeError`!

Practical Code Examples

```
JAVASCRIPT

const note = "Emails: support@cloud.com and billing@cloud.com";

// 1. Without /g: Grabs first email AND unpacks username + domain!
const single = note.match(/[a-z0-9]+@[a-z0-9.]+/i);
console.log(single[0]); // "support@cloud.com" (Full match)
console.log(single[1]); // "support" (Group 1: username)
console.log(single[2]); // "cloud.com" (Group 2: domain)

// 2. With /g: Vacuum mode (grabs all emails):
const all = note.match(/[a-z0-9]+@[a-z0-9.]+/gi) || [];
console.log(all); // ["support@cloud.com", "billing@cloud.com"]
```

8. `str.matchAll(regex)` — The Full Medical X-Ray Scanner 🏥 (ES2020)

What is it in Plain English?

What if you want **ALL matches** across the text, BUT you also need the **capture groups** `()` for each one? Before ES2020, this was painful. `matchAll()` is the modern champion that does both cleanly!

The One Strict Invariant:

⚠️ **The Global Flag Rule**: The regex passed to `matchAll()` **MUST have the `/g` flag**, or JavaScript throws an immediate `TypeError`!

Practical Code Examples

```
JAVASCRIPT

const markdown = "Check [Google](https://google.com) and [GitHub](https://github.com)";
const linkRegex = /\[([^\]]+)\]\((https?:\/\/[^\)]+)\)/g;

// Spreading into an Array to inspect every match with full capture groups:
const results = [...markdown.matchAll(linkRegex)];

for (const match of results) {
  const [full, text, url] = match;
  console.log(`Found link: "${text}" -> ${url} at position ${match.index}`);
}

// Found link: "Google" -> https://google.com at position 6
// Found link: "GitHub" -> https://github.com at position 36
```

09. Transformation Methods

Transformation methods generate a modified version of an existing string.

💡 **Core Invariant:** Because JavaScript strings are **immutable**, transformations **never alter the original string in memory**. They allocate and hand you a **brand new string**.

1. Case Conversion — The Whisperer 🗯️ & The Megaphone 📢

Methods & What They Do

- `str.toLowerCase()`: The Whisperer 🗯️ — turns every character into lowercase (`"HELLO"` -> `"hello"`).
- `str.toUpperCase()`: The Megaphone 📢 — turns every character into uppercase (`"hello"` -> `"HELLO"`).

⚠️ The Famous "Turkish-I" Passport Bug in Production

In the English alphabet, the capital of `'i'` is `'I'`. However, in Turkish and Azeri, there are **two different letters I**:

- Dotted `İ` (uppercase) / `ı` (lowercase)
- Dotless `I` (uppercase) / `i` (lowercase)

If your app runs on a user's phone in Turkey:

JAVASCRIPT

```
// English / Default:
console.log("title".toUpperCase()); // "TITLE"

// Turkish:
console.log("title".toLocaleUpperCase("tr")); // "TİTLE" (Notice the dotted İ!)
console.log("TİTLE".toLocaleLowerCase("tr")); // "tİtle" (Notice the dotless ı!)
```

💡 Senior Production Rule:

- For **internal code, JSON keys, API routes, and database tokens**: ALWAYS use standard `.toLowerCase()` or `.toUpperCase()`.
- For **human-facing text displayed to international users**: Use `.toLocaleLowerCase(userLocale)`.

2. Trimming Whitespace — The Beard Trimmer ✂️

Methods & What They Do

When users type into forms, they accidentally add extra spaces at the beginning or end. Trimming methods shave away that invisible fuzz while leaving the middle untouched:

- `str.trim()`: Shaves whitespace from **both** the start and the end.
- `str.trimStart()`: Shaves whitespace only from the **start** (left side).
- `str.trimEnd()`: Shaves whitespace only from the **end** (right side).

What Counts as Whitespace?

It doesn't just trim normal spaces (`" "`). It strips tabs (`\t`), newlines (`\n`), carriage returns (`\r`), non-breaking spaces (`\u00A0`), and Unicode spaces!

Practical Code Examples

JAVASCRIPT

```
function cleanEmail(input) {  
  return input.trim().toLowerCase();  
}  
  
console.log(cleanEmail("  Ayush.Dev@Domain.com \n "));  
// "ayush.dev@domain.com" (Clean and ready for the database!)
```

3. Padding — The Pillow Spacers 🛏️

Methods & What They Do

Imagine you need a string to be a specific length (say, 5 characters wide). Padding methods stuff extra "pillows" (spaces or characters) on the left or right until the string reaches your desired target length:

- `str.padStart(targetLength, padString)` : Adds padding to the **front (left)**.
- `str.padEnd(targetLength, padString)` : Adds padding to the **back (right)**.

Real-World Dev Use Cases

- **Zero-Padding Invoice Numbers:** Turning `7` into `"0007"`.
- **Masking Sensitive Payment Cards:** Turning `"1234"` into `"*****1234"`.
- **Aligning CLI Terminal Tables:** Making columns line up neatly.

Practical Code Examples

JAVASCRIPT

```
// 1. Digital Clock Formatter (e.g. "09:05:02"):  
function formatTime(hours, minutes, seconds) {  
  const h = String(hours).padStart(2, "0");  
  const m = String(minutes).padStart(2, "0");  
  const s = String(seconds).padStart(2, "0");  
  return `${h}:${m}:${s}`;  
}  
console.log(formatTime(9, 5, 2)); // "09:05:02"  
  
// 2. Masking Credit Card:  
const last4 = "8912";  
console.log(last4.padStart(16, "*")); // "*****8912"  
  
// 3. CLI Table Column Alignment:  
console.log("Status".padEnd(12, ".") + " " + "ONLINE".padStart(8, " "));  
console.log("Database".padEnd(12, ".") + " " + "ONLINE".padStart(8, " "));  
// Status..... ONLINE  
// Database....  ONLINE
```

4. Repeating — The Rubber Stamp Machine 🖨️

What is it in Plain English?

`str.repeat(count)` takes your string and stamps **N carbon copies** of it back-to-back into one string.

Practical Code Examples

JAVASCRIPT

```
// 1. Terminal Divider Line:
console.log("=".repeat(30));
console.log("  APPLICATION LOGS");
console.log("=".repeat(30));

// 2. Code Indentation Generator:
function indent(level) {
  return " ".repeat(level);
}
console.log(indent(0) + "root");
console.log(indent(1) + "src");
console.log(indent(2) + "index.js");
```

5. String Replacement — The Single-Target Sniper 🎯

What is it in Plain English?

`replace(pattern, replacement)` searches for a pattern and swaps it with replacement text.

⚠️ The #1 Trap That Bites Every Junior Developer:

If you pass a **string** or a **RegExp without `/g`**, **ONLY THE FIRST MATCH IS REPLACED!**

JAVASCRIPT

```
const fruit = "banana";

// ❌ Trap: Only replaces the FIRST 'a'!
console.log(fruit.replace("a", "o")); // "bonana" (NOT "bonono")

// ✅ To replace all using regex, add the global flag /g:
console.log(fruit.replace(/a/g, "o")); // "bonono"
```

Special Replacement Tokens

When replacing, JavaScript gives you handy shortcuts:

- `$&` : Injects the matched word itself (great for wrapping text in brackets or bold tags!).
- `$1`, `$2` : Injects the 1st or 2nd parenthesized capture group.

Practical Code Examples

JAVASCRIPT

```
// 1. Swapping First and Last Names:
const author = "Turing, Alan";
const fixed = author.replace(/(\w+),\s+(\w+)/, "$2 $1");
console.log(fixed); // "Alan Turing"

// 2. Wrapping keywords using $&:
const alertMsg = "Server status: critical error";
console.log(alertMsg.replace(/critical/, "**[$&]**"));
// "Server status: **[critical]** error"

// 3. Dynamic Replacer Function (kebab-case to camelCase):
function toCamelCase(str) {
  return str.replace(/-([a-z])/g, (match, letter) => letter.toUpperCase());
}
console.log(toCamelCase("font-size-large")); // "fontSizeLarge"
```

6. Global String Replacement — The Mass Renamer 🛠️ (ES2021)

What is it in Plain English?

Before ES2021, if you wanted to replace *every* comma or *every* slash in a string, you had to either write a regular expression with escaping (`/\\//g`) or use `split().join()` .

`replaceAll()` solves this cleanly: it replaces **every single occurrence of a literal string** without needing any regular expressions!

The One Strict Rule:

If you pass a Regular Expression to `replaceAll()` , it **MUST have the `/g` flag**, otherwise JavaScript throws a `TypeError` !

Practical Code Examples

JAVASCRIPT

```
const rawQuery = "apples&oranges&bananas&grapes";

// Replaces EVERY '&' with ' + ' cleanly:
console.log(rawQuery.replaceAll("&", " + "));
// "apples + oranges + bananas + grapes"

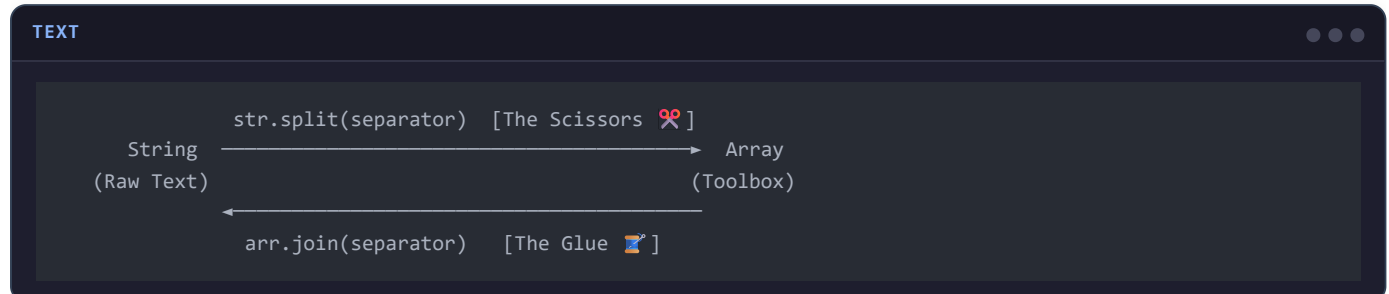
// Normalizing Windows file path slashes:
const winPath = "C:\\Users\\ayush\\documents\\report.pdf";
console.log(winPath.replaceAll("\\", "/"));
// "C:/Users/ayush/documents/report.pdf"
```

10. Split & Join: The String ⇄ Array Bridge

In JavaScript, **Strings** and **Arrays** are close cousins, but they have completely different superpowers:

- **Strings are "Locked Down" (Immutable):** Once created, you cannot swap characters, reorder them, sort them, or pop them off. A string is fixed in memory.
- **Arrays are "Flexible Toolkits" (Mutable):** Arrays come packed with dozens of powerful manipulation tools: `.reverse()`, `.sort()`, `.filter()`, `.map()`, `.reduce()`, and `.splice()`.

So what do you do when you need to sort letters, reverse words, clean up a comma-separated list, or process data? You build a **two-way bridge** between Strings and Arrays:



- `str.split()` is your **scissors** ✂: It cuts a string apart into an Array of pieces.
- `arr.join()` is your **glue** 🧴: It stitches an Array of pieces back together into a single String.

1. `str.split([separator[, limit]])` — The Scissors ✂

What is it in Plain Layman's Terms?

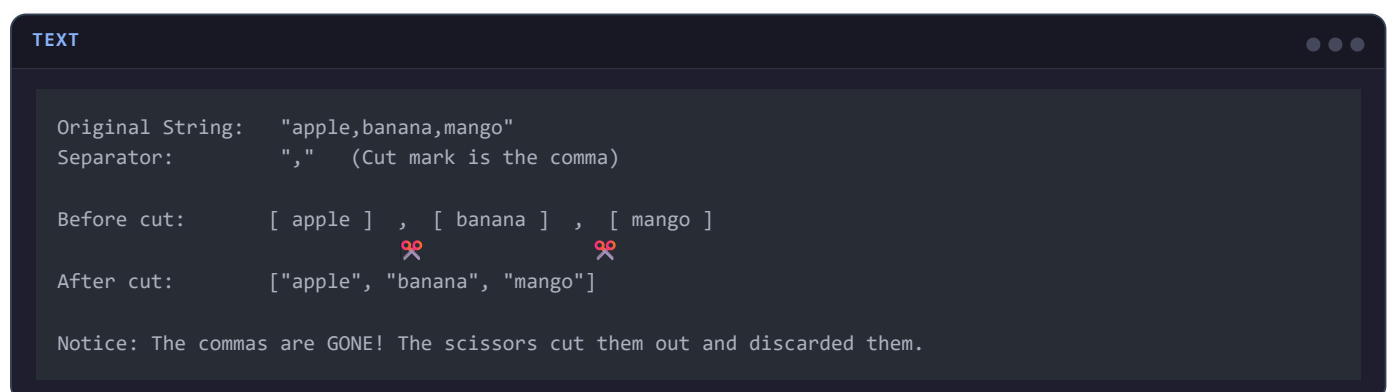
Imagine your string is a paper ribbon or a loaf of bread. Calling `.split()` is like picking up a pair of **scissors**.

The `separator` tells JavaScript:

"Look through this string. Every time you see this mark, CUT right here and throw the mark away!"

JavaScript cuts the string wherever that mark appears, collects all the loose slices into an **Array (a box of pieces)**, and hands that array to you.

Visualizing the Cut



Formal Syntax

JAVASCRIPT

```
str.split()           // No arguments
str.split(separator)  // Cut at separator
str.split(separator, limit) // Cut at separator, but keep only 'limit' pieces
```

The 4 Everyday Ways to Use split()

Case 1: Cut by a character or word (str.split(","), str.split(" "), str.split("-"))

Whenever you have structured data (like CSV files, dates, or sentences), use the character that separates each item:

JAVASCRIPT

```
// Cutting by comma:
const tags = "javascript,react,nodejs";
console.log(tags.split(","));
// ["javascript", "react", "nodejs"]

// Cutting by space (splits a sentence into words!):
const sentence = "Learning JavaScript is fun";
console.log(sentence.split(" "));
// ["Learning", "JavaScript", "is", "fun"]

// Cutting by hyphen (splits a date):
const date = "2026-12-31";
console.log(date.split("-"));
// ["2026", "12", "31"]
```

Case 2: Cut by empty string "" (Chop between EVERY single letter!)

If you give an empty string `""` as the separator, you are telling JavaScript:

"There is nothing between the cuts—so cut between EVERY single character!"

JAVASCRIPT

```
const word = "code";
console.log(word.split(""));
// ["c", "o", "d", "e"]
```

⚠️ **The Emoji / Surrogate Pair Warning:** JavaScript stores characters in 16-bit units. Complex characters like emojis (🔥🌍) take two 16-bit units. If you use `"".split("")`, the scissors cut the emoji right down the middle, producing broken symbols:

JAVASCRIPT

```
"🔥🌍".split(""); // ["\uD83D", "\uDC4B"] <-- Broken half-emojis!
```

✅ **Safe Modern Alternative:** Use the spread operator `[...str]` or `Array.from(str)`:

JAVASCRIPT

```
console.log([..."🔥🌍"]); // ["🔥", "🌍"] <-- Perfect!
```

Case 3: No separator passed at all (`str.split()`)

What if you call `str.split()` without passing any arguments? Mental model: *You gave JavaScript scissors, but didn't tell it where to cut!* So JavaScript makes **zero cuts** and simply places the entire unbroken string inside an array:

JAVASCRIPT

```
const text = "hello";
console.log(text.split());
// ["hello"] (An array with just 1 element)
```

Case 4: The separator does not exist in the string

What if you tell it to cut at `","`, but the string only contains commas?

JAVASCRIPT

```
const data = "a,b,c";
console.log(data.split(","));
// ["a,b,c"]
```

Because the scissors couldn't find any `,` to cut at, nothing gets cut. You get back the whole string inside a 1-item array.

What Does the Second Argument (`limit`) Do?

The `limit` argument tells the scissors:

"Stop cutting once you have collected this many slices. Throw the rest away!"

JAVASCRIPT

```
const items = "one,two,three,four,five";

// Only take the first 2 slices:
console.log(items.split(",", 2));
// ["one", "two"]

// Only take the first 3 slices:
console.log(items.split(",", 3));
// ["one", "two", "three"]
```

💡 Real-world use: Splitting a key-value pair where the value might contain equal signs:

JAVASCRIPT

```
const header = "auth=bearer=token123=extra";
const [key, firstVal] = header.split("=", 2);
console.log(key); // "auth"
console.log(firstVal); // "bearer=token123=extra"
```

💡 The Secret Weapon: Keeping Delimiters with Parentheses ()

Normally, `.split()` discards the separator (the commas or spaces vanish). **BUT** if your separator is a Regular Expression with capturing parentheses `()`, JavaScript keeps the cut marks inside the array!

JAVASCRIPT

```
// Without parentheses: operators disappear
console.log("10+20-5".split(/[+-]/));
// ["10", "20", "5"]

// WITH parentheses: operators are PRESERVED in the array!
console.log("10+20-5".split(/[+-]/));
// ["10", "+", "20", "-", "5"]
```

This is how code interpreters and math calculators break math formulas into tokens without losing the `+` or `-` symbols!

2. `arr.join([separator])` — The Glue 🧰

What is it in Plain Layman's Terms?

`join()` is the exact mirror opposite of `split()`. If `split()` was scissors, `join()` is a **tube of glue** or a needle and thread.

You have an Array of separate pieces sitting in a box. You want to stick them back together into **one continuous string**. The `separator` argument is the glue you place **in between** every pair of items.

Visualizing the Glue

```
TEXT

Original Array:  ["apple", "banana", "mango"]
Glue Separator:  " - "

Stitching together:
"apple"  +  " - "  +  "banana"  +  " - "  +  "mango"
  ↑        ↑        ↑        ↑        ↑
Item 1    Glue    Item 2    Glue    Item 3

Result String:   "apple - banana - mango"
```

💡 The Golden Rule of Glue:

Glue only goes BETWEEN items. There is NEVER glue before the first item, and NEVER glue after the last item!

- 3 items in array → 2 glue spots.
- 2 items in array → 1 glue spot.
- 1 item in array → 0 glue spots (`["solo"].join("-")` produces `"solo"`).
- 0 items (empty array `[]`) → produces an empty string `""` .

Formal Syntax

```
JAVASCRIPT

arr.join()           // Defaults to comma ","
arr.join(separator)  // Uses custom string as glue
```

The 3 Most Common Ways to Use join()

1. Default Glue: The Comma ,

If you do not pass any argument, JavaScript uses a comma `,` by default:

```
JAVASCRIPT

const fruits = ["apple", "banana", "orange"];
console.log(fruits.join());
// "apple,banana,orange"
```

2. Invisible Glue: Empty String "" (Glued Directly Together)

If you pass an empty string `""` , the items are glued directly together with zero space between them:

JAVASCRIPT

```
const letters = ["c", "o", "d", "e"];
console.log(letters.join(""));
// "code"
```

(This is the most common way to turn an array of letters back into a word!).

3. Custom Glue (Spaces, Slashes, Newlines, Hyphens, Arrows)

You can use whatever glue you want:

JAVASCRIPT

```
const pathParts = ["users", "ayush", "documents", "reports"];

// Web URL path glue ("/"):
console.log(pathParts.join("/"));
// "users/ayush/documents/reports"

// Breadcrumb arrow glue (" > "):
console.log(pathParts.join(" > "));
// "users > ayush > documents > reports"

// Newline glue ("\n" - makes a multi-line list!):
const todoList = ["1. Buy milk", "2. Code JavaScript", "3. Sleep"];
console.log(todoList.join("\n"));
// 1. Buy milk
// 2. Code JavaScript
// 3. Sleep
```

3. The Classic "Washing Machine" Recipe

How professional developers use `split` and `join` together to solve real-world coding challenges:

TEXT

```
Raw String → .split() → Array (do array operations) → .join() → Final String
```

Recipe Walkthrough 1: Reverse a String

Strings don't have a `.reverse()` method. But Arrays do!

JAVASCRIPT

```
function reverseString(text) {  
  // 1. Cut string into letters (Array)  
  const letters = text.split(""); // ["h", "e", "l", "l", "o"]  
  
  // 2. Use Array superpower: reverse  
  letters.reverse(); // ["o", "l", "l", "e", "h"]  
  
  // 3. Glue letters back together with invisible glue  
  return letters.join(""); // "olleh"  
}  
  
console.log(reverseString("cloud")); // "duolc"
```

Recipe Walkthrough 2: Sort the Letters of a Word Alphabetically

JAVASCRIPT

```
function sortLetters(word) {  
  return word  
    .split("") // ["z", "a", "b"]  
    .sort() // ["a", "b", "z"]  
    .join(""); // "abz"  
}  
  
console.log(sortLetters("javascript")); // "aacijprstv"
```

Recipe Walkthrough 3: Check If a Word is a Palindrome (Reads same backward)

JAVASCRIPT

```
function isPalindrome(word) {  
  const clean = word.toLowerCase();  
  const reversed = clean.split("").reverse().join("");  
  return clean === reversed;  
}  
  
console.log(isPalindrome("racecar")); // true  
console.log(isPalindrome("banana")); // false
```

4. 5 Graded Real-World Production Examples

Example 1: Parsing CSV Records into Clean Data

When reading CSV files or spreadsheet exports, fields often have messy surrounding spaces:

JAVASCRIPT

```
function parseCSVRow(rowText) {  
  // 1. Cut at each comma  
  // 2. Trim whitespace from each field  
  // 3. Filter out any accidentally empty fields  
  return rowText  
    .split(",")  
    .map(cell => cell.trim())  
    .filter(cell => cell.length > 0);  
}  
  
const rawRow = "  Ayush  , Cloud Engineer ,   Mumbai  , 2026  ";  
console.log(parseCSVRow(rawRow));  
// ["Ayush", "Cloud Engineer", "Mumbai", "2026"]
```

Example 2: URL Query String to Object Parser

Converting `"?page=2&sort=asc&filter=active"` into a JavaScript `{ key: value }` object:

JAVASCRIPT

```
function parseQuery(queryString) {  
  // Remove Leading '?' if present  
  const clean = queryString.startsWith("?") ? queryString.slice(1) : queryString;  
  if (!clean) return {};  
  
  const params = {};  
  
  // Step 1: Split each key-value pair at '&'  
  const pairs = clean.split("&"); // ["page=2", "sort=asc", "filter=active"]  
  
  // Step 2: Split each pair at '='  
  for (const pair of pairs) {  
    const [key, val] = pair.split("=");  
    params[decodeURIComponent(key)] = decodeURIComponent(val ?? "");  
  }  
  
  return params;  
}  
  
console.log(parseQuery("?product=laptop&category=tech&inStock=true"));  
// { product: "laptop", category: "tech", inStock: "true" }
```

Example 3: Whitespace-Resilient Word Counter

Users often type multiple spaces, tabs, or newlines in text areas. Splitting on `" "` alone fails because it leaves empty string elements. Splitting with a regex `/\s+/` (one or more spaces) fixes it:

JAVASCRIPT

```
function countWords(text) {
  const trimmed = text.trim();
  if (!trimmed) return 0;

  // \s+ means: cut at 1 or more spaces, tabs, or newlines
  const words = trimmed.split(/\s+/);
  return words.length;
}

const messyText = " Hello  there! \n\t Welcome to the team. ";
console.log(countWords(messyText)); // 6 words!
```

Example 4: Formatting a Slug for Blog URLs

Turn any blog title into a clean, SEO-friendly URL slug:

JAVASCRIPT

```
function createSlug(title) {
  return title
    .toLowerCase()
    .trim()
    .split(/[^\a-z0-9]/) // Cut at any non-alphanumeric character
    .filter(Boolean)     // Remove empty slices
    .join("-");          // Glue together with hyphens!
}

console.log(createSlug("10 Tips for Mastering JavaScript in 2026!"));
// "10-tips-for-mastering-javascript-in-2026"
```

Example 5: Preserving Math Operators During Tokenization

Using regex capturing groups `()` so the operators don't get thrown away:

JAVASCRIPT

```
function tokenizeMath(mathExpression) {
  // Notice the capturing parentheses around ([+|-*/])
  return mathExpression
    .split(/([+|-*/])/)
    .map(token => token.trim())
    .filter(token => token.length > 0);
}

console.log(tokenizeMath("100 + 45 * 2 - 8"));
// ["100", "+", "45", "*", "2", "-", "8"]
```

Quick Cheat Sheet: split vs join

TEXT		
Feature	str.split(separator)	arr.join(separator)
Mental Model	✂ Scissors (Cutting text apart)	🔗 Glue (Stitching pieces back)
Input Type	String	Array
Output Type	Array of substrings	Single combined String
Default if empty	[str] (whole string in 1 array)	Comma-separated: "a,b,c"
With ""	Cuts into individual letters	Glues letters with zero space
Where mark goes	Discarded (unless using regex ())	Placed only BETWEEN elements

11. Template Literals & Tagged Templates

Core Template Literal Features

1. **Interpolation:** Embedded expressions via `${expression}`.
2. **Multiline strings:** Retains formatting and newlines naturally without `\n`.
3. **Escaping backticks:** `\``.

```
JAVASCRIPT
const user = "Ayush";
const unread = 5;
const msg = `Hello ${user},
You have ${unread} unread notifications.`;
```

Tagged Template Functions

A tagged template is a function invoked with a template literal without parentheses:

```
JAVASCRIPT
function tag(strings, ...values) {
  return strings.reduce((acc, str, i) => acc + str + (values[i] ?? ""), "");
}
```

🔒 Critical Security Rule: SQL Injection Defense

⚠ **Common Junior Mistake:** Trying to build a custom string-escaping function with tagged templates and trusting it as SQL injection defense.

- **The Flaw:** Character escaping is fragile, database-dialect specific, and vulnerable to character set attacks.
- **The Senior Standard:** In production backends (PostgreSQL/Node.js), **always use Parameterized Queries / Prepared Statements:**

JAVASCRIPT

```
// ❌ DANGEROUS: String interpolation / custom escaping
// const q = `SELECT * FROM files WHERE user_id = '${userId}'`;

// ✅ SECURE: Parameterized Query using pg client
// await db.query('SELECT * FROM files WHERE user_id = $1', [userId]);
```

String.raw

Treats backslashes as literal raw characters without parsing escape codes:

JAVASCRIPT

```
const winPath = String.raw`C:\new\text\report.pdf`;
console.log(winPath); // "C:\new\text\report.pdf" (\n is not a newline!)
```

12. Regular Expressions — Beginner

1. What is a Regular Expression?

A **Regular Expression (RegExp)** is a pattern describing a sequence of characters, used for searching, matching, validating, and replacing text.

2. Creating a RegExp

JAVASCRIPT

```
// 1. Literal Syntax (compiled at script load time, preferred for static patterns)
const re1 = /pdf/i;

// 2. RegExp Constructor (compiled at runtime, required for dynamic patterns)
const searchWord = "pdf";
const re2 = new RegExp(searchWord, "i");
```

3. Literal Characters vs Metacharacters

- **Literal:** Characters matching themselves: `/cat/` matches `"cat"`.
- **Metacharacters:** Special characters with engine meaning: `.`, `*`, `+`, `?`, `^`, `$`, `[`, `]`, `(`, `)`, `{`, `}`, `|`, `\`. To match a literal dot `.`, escape with backslash: `/report\.pdf/`.

4. Character Classes

- `[abc]`: Any one of 'a', 'b', or 'c'.
- `[a-z]`: Any lowercase letter from 'a' to 'z'.
- `[A-Z]`: Any uppercase letter.
- `[0-9]`: Any digit from 0 to 9.
- `\d`: Digit shorthand (`[0-9]`).
- `\D`: Non-digit shorthand (`[^0-9]`).

- `\w` : Word character (`[a-zA-Z0-9_]`).
- `\W` : Non-word character.
- `\s` : Whitespace character (space, tab, newline).
- `\S` : Non-whitespace character.
- `.` : Any character except newline (unless `s` flag is on).

5. Negated Character Classes

- `[^0-9]` : Matches any character that is NOT a digit.
- `[^aeiou]` : Matches any non-vowel.

6. Quantifiers (How Many Times?)

- `*` : 0 or more times (greedy).
- `+` : 1 or more times (greedy).
- `?` : 0 or 1 time (optional).
- `{n}` : Exactly `n` times.
- `{n,}` : At least `n` times.
- `{n,m}` : Between `n` and `m` times inclusive.

7. Greedy vs Lazy (Reluctant) Quantifiers

By default, quantifiers are **greedy**—they match as many characters as possible. Append `?` to make a quantifier **lazy**—matching as few characters as possible:

```
JAVASCRIPT

const html = "<div>First</div><div>Second</div>";

// Greedy (+ matches up to the LAST </div>!):
console.log(html.match(/<div>.*</div>/)[0]);
// "<div>First</div><div>Second</div>"

// Lazy (.* stops at the FIRST </div>!):
console.log(html.match(/<div>.*?</div>/)[0]);
// "<div>First</div>"
```

8. Anchors: Start (^) and End (\$)

- `^file` : String must start with "file".
- `\.pdf$` : String must end with ".pdf".
- `^\d{4}$` : String must consist of exactly 4 digits and nothing else.

13. Regular Expressions — Intermediate

1. Groups & Capturing ((...))

Parentheses create a **capturing group**:

JAVASCRIPT

```
const fileStr = "report_2026.pdf";
const match = fileStr.match(/([a-z]+)_([d]{4})\.([a-z]+)/);

console.log(match[0]); // "report_2026.pdf" (Full match)
console.log(match[1]); // "report" (Group 1)
console.log(match[2]); // "2026" (Group 2)
console.log(match[3]); // "pdf" (Group 3)
```

2. Non-Capturing Groups ((?:...))

When you need grouping for alternation or quantifiers without capturing:

JAVASCRIPT

```
// Matches "https" or "http", but does not capture it:
const pattern = /(?:https|http):\\/drive\\.com/;
```

3. Named Capture Groups ((?<name>...)) (ES2018)

Gives capture groups human-readable dictionary names:

JAVASCRIPT

```
const log = "2026-09-12 [ERROR] Database timeout";
const re = /(?<date>\d{4}-\d{2}-\d{2}) \[(?<level>[A-Z]+)\] (?<message>.+)/;
const result = log.match(re);

console.log(result.groups.date); // "2026-09-12"
console.log(result.groups.level); // "ERROR"
console.log(result.groups.message); // "Database timeout"
```

4. Word Boundaries (\b)

Matches a word boundary position (transition between `\w` and `\W`):

JAVASCRIPT

```
const text = "The cat scattered the food.";
console.log(text.replace(/cat/, "dog")); // "The dog scattered..." (Accidental match in scattered!)
console.log(text.replace(/\bcat\b/, "dog")); // "The dog scattered the food." (Only exact word "cat!")
```

5. Regular Expression Flags

- `g`: Global (matches all occurrences).
- `i`: Case-insensitive.
- `m`: Multiline (`^` and `$` match start/end of each line, not just full string).
- `s`: DotAll (`.` matches newlines as well).
- `u`: Unicode (enables full Unicode code point support).
- `y`: Sticky (matches strictly from `lastIndex`).

- `d`: Indices (provides start and end indices of capture groups in `.indices`).

14. Regular Expressions — Advanced (Lookarounds & ReDoS)

1. Lookahead Assertions

Checks if a pattern is followed by something without including it in the match:

- **Positive Lookahead** `(?=...)`: Must be followed by pattern.

JAVASCRIPT

```
// Match password with at least one number:
const hasNumber = /^(?=.*\d)/;

// Extract number before currency:
const str = "Total: 50 USD";
const num = str.match(/\d+(?=\s*USD)/);
console.log(num[0]); // "50"
```

- **Negative Lookahead** `(?!...)`: Must NOT be followed by pattern.

JAVASCRIPT

```
// Match "drive" only when NOT followed by ".com":
const re = /drive(?!\.com)/;
```

2. Lookbehind Assertions (ES2018)

Checks if a pattern is preceded by something:

- **Positive Lookbehind** `(?<=...)`: Must be preceded by pattern.

JAVASCRIPT

```
const cost = "Price: $150";
const amount = cost.match(/(?<=\$)\d+/);
console.log(amount[0]); // "150"
```

- **Negative Lookbehind** `(?<!\...)`: Must NOT be preceded by pattern.

JAVASCRIPT

```
// Numbers not preceded by $:
const freeNumber = /^(?<!\$)\d+/;
```

3. Backreferences (`\1`, `\2`)

Matches the exact same text matched by an earlier capturing group:

JAVASCRIPT

```
// Find duplicated consecutive words ("the the"):
const findDups = /\b(\w+)\s+\1\b/i;
console.log("This is the the file".match(findDups)[0]); // "the the"
```

4. Catastrophic Backtracking & ReDoS (Regular Expression Denial of Service)

⚠ Critical Security Warning: When a regex contains nested quantifiers on overlapping patterns (e.g. `/(a+)+$/`), an unmatched input causes the engine to explore $O(2^n)$ exponential paths, freezing the single-threaded Node.js event loop!

JAVASCRIPT

```
// ❌ CATASTROPHIC BACKTRACKING:
// const evilRegex = /(a+)+$/;
// evilRegex.test("aaaaaaaaaaaaaaaaaaaaaaaaaab"); // FREEZES CPU!

// ✅ SAFE PATTERN: Avoid nested greedy quantifiers:
const safeRegex = /^[a-zA-Z0-9]+$/;
```

15. Unicode Fundamentals

Computers only understand numbers (bits). A **character encoding** is a standard table mapping human characters to numeric IDs called **code points**.

- **ASCII** (1963): 7-bit system (0 to 127). English alphabet only.
- **Unicode** (1991 - Present): Universal character set assigning a unique code point to every character in all human languages, mathematical symbols, and emojis.
- Code point notation: `U+0041` (Letter 'A'), `U+1F4C1` (📁 Folder emoji).

16. UTF-16 & Code Units

JavaScript natively stores strings as **UTF-16**:

- Memory is organized in **16-bit code units** (values `0` to `65535` or `0x0000` to `0xFFFF`).
- The **Basic Multilingual Plane (BMP)** covers characters up to `U+FFFF` using a single 16-bit code unit.
- Characters above `U+FFFF` are called **Supplementary Planes** and require two 16-bit units.

17. Code Points & Surrogate Pairs

A **Surrogate Pair** is a pair of 16-bit code units used in UTF-16 to encode a single 32-bit Unicode code point above `0xFFFF`:

- **High Surrogate:** Range `0xD800` to `0xDBFF` (55296 to 56319).
- **Low Surrogate:** Range `0xDC00` to `0xDFFF` (56320 to 57343).

JAVASCRIPT

```
const emoji = "👨‍👩‍👧‍👦"; // Code point U+1F4C1 (128209)

// 1. Length reflects 16-bit code units:
console.log(emoji.length); // 2!

// 2. charCodeAt returns individual 16-bit surrogate units:
console.log(emoji.charCodeAt(0)); // 55357 (0xD83D - High surrogate)
console.log(emoji.charCodeAt(1)); // 56513 (0xDCC1 - Low surrogate)

// 3. codePointAt returns the unified 32-bit code point:
console.log(emoji.codePointAt(0)); // 128209 (0x1F4C1)

// 4. Reconstructing characters from numbers:
console.log(String.fromCharCode(128209)); // Corrupted output!
console.log(String.fromCodePoint(128209)); // "👨‍👩‍👧‍👦" (Correct!)
```

18. Grapheme Clusters & Intl.Segmenter

The Multi-Codepoint Compound Emoji Trap

Many emojis are formed by combining multiple code points using a **Zero Width Joiner (ZWJ, \u200D)**:

- 👨‍👩‍👧‍👦 (Family) is constructed of 4 individual human emojis and 3 ZWJs:

JAVASCRIPT

```
const family = "👨‍👩‍👧‍👦";

console.log(family.length); // 11 code units!
console.log([...family].length); // 7 code points! (Spread fails on compound emojis!)
```

The Solution: Grapheme Clusters via Intl.Segmenter (ES2022)

A **grapheme cluster** is what a human being perceives as a single visual character on screen.

JAVASCRIPT

```
const segmenter = new Intl.Segmenter("en", { granularity: "grapheme" });
const graphemes = [...segmenter.segment(family)];

console.log(graphemes.length); // 1! (Exactly 1 perceived human character!)
```

19. Unicode Normalization: .normalize()

In Unicode, accented characters can be represented in two distinct forms:

- Composed (NFC)**: A single pre-combined code point (e.g. é is \u00E9).
- Decomposed (NFD)**: The letter e followed by a combining acute accent ´ (e.g. \u0065\u0301).

JAVASCRIPT

```
const fileNFC = "caf\u00E9.pdf";
const fileNFD = "cafe\u0301.pdf";

console.log(fileNFC === fileNFD); // false! Look identical, but different byte sequences!

// Normalization brings them into canonical composed form:
console.log(fileNFC.normalize("NFC") === fileNFD.normalize("NFC")); // true!
```

Normalization Forms:

- **NFC**: Canonical Composition (Standard for web & file systems).
- **NFD**: Canonical Decomposition.
- **NFKC**: Compatibility Composition.
- **NFKD**: Compatibility Decomposition.

20. Well-Formed Strings: `.isWellFormed()` & `.toWellFormed()` (ES2024)

If a surrogate pair is cut in half, it produces an orphan **lone surrogate**, which causes crashes in database drivers and `encodeURIComponent`.

JAVASCRIPT

```
const valid = "file_📁.pdf";
const corrupt = "file_\uD83D.pdf"; // Missing low surrogate half!

console.log(valid.isWellFormed()); // true
console.log(corrupt.isWellFormed()); // false!

// toWellFormed replaces lone surrogates with U+FFFD ():
const safeStr = corrupt.toWellFormed();
console.log(safeStr); // "file_📁.pdf"
```

21. Internationalization

The Turkish `i` Bug

In Turkish (`tr-TR`), the uppercase of `i` is `İ` (dotted), and lowercase of `I` is `ı` (dotless):

JAVASCRIPT

```
console.log("TITLE".toLocaleLowerCase("en-US")); // "title"
console.log("TITLE".toLocaleLowerCase("tr-TR")); // "title" (Dotless i!)
```

Natural Numeric Sorting with `localeCompare()` & `Intl.Collator`

Normal `.sort()` sorts alphabetically by ASCII bytes, placing `"file10"` before `"file2"`. Use `localeCompare` with `{ numeric: true }`:

JAVASCRIPT

```
const files = ["file10.txt", "file2.txt", "file1.txt"];

files.sort((a, b) => a.localeCompare(b, undefined, { numeric: true }));

console.log(files); // ["file1.txt", "file2.txt", "file10.txt"]
```

22. Strings + Arrays

Patterns: String $\rightleftharpoons$ Array Processing

JAVASCRIPT

```
// 1. Character frequency counter:
function getFrequency(str) {
  return [...str].reduce((acc, char) => {
    acc[char] = (acc[char] ?? 0) + 1;
    return acc;
  }, {});
}
console.log(getFrequency("storage"));

// 2. Capitalize every word:
function capitalizeWords(sentence) {
  return sentence
    .split(" ")
    .map(word => word.charAt(0).toUpperCase() + word.slice(1).toLowerCase())
    .join(" ");
}
console.log(capitalizeWords("full stack cloud platform"));
// "Full Stack Cloud Platform"
```


25. Encoding & Escaping

encodeURIComponent() vs encodeURI()

- `encodeURI(fullUrl)` : Encodes a full URL, preserving protocol and path delimiters (`:`, `/`, `?`, `&`, `#`).
- `encodeURIComponent(val)` : Encodes a query parameter or path segment, encoding **everything** except unreserved characters (`A-Za-z0-9-._~!*'()`).

JAVASCRIPT

```
const param = "Annual Report & Budget / 2026";

console.log(encodeURI(param));
// "Annual%20Report%20&%20Budget%20/%202026" (Preserved & and /)

console.log(encodeURIComponent(param));
// "Annual%20Report%20%26%20Budget%20%2F%202026" (Safely encoded & and /)
```

HTML Escaping

JAVASCRIPT

```
function escapeHtml(str) {
  return str
    .replaceAll("&", "&")
    .replaceAll("<", "<")
    .replaceAll(">", ">")
    .replaceAll("'", "'")
    .replaceAll('"', '"');
}
```

26. String Algorithms — Beginner (1 to 23)

Mastering string manipulation is foundational for technical interviews and day-to-day engineering. In technical interviews, interviewers typically evaluate you on two distinct dimensions:

- Idiomatic JavaScript (With Built-in Methods):** Demonstrating deep fluency with high-level language capabilities (`.split()`, `.reverse()`, `.replaceAll()`, `Set`, `regex`).
- Algorithmic Fundamentals (Without Built-in Methods):** Solving the exact same problem from scratch using raw loops, pointers, index arithmetic, and ASCII codes without relying on helper abstractions.

Below are the **23 essential beginner string algorithms**. Every problem includes:

-  **Problem Description:** Clear breakdown of what is expected.
-  **Examples:** Concrete inputs, outputs, and edge cases.
-  **Constraints:** Bounds and edge conditions.
-  **Starter Code:** A ready-to-use function template so you can attempt solving it first.
-  **Collapsible Solution Toggle:** Contains **TWO distinct solutions** (Solution 1: With Built-in Methods, Solution 2: Without Built-in Methods / From Scratch), followed by complexity analysis for both!

Problem 1: Reverse a String (Easy)

Problem Description

Write a function `reverseString(str)` that takes a string `str` and returns a new string containing the same characters in exact reversed order.

Examples

- **Example 1:**

- **Input:** `str = "cloud"`
- **Output:** `"duolc"`
- **Explanation:** The characters read from right to left are `'d'`, `'l'`, `'o'`, `'u'`, `'c'`.

- **Example 2:**

- **Input:** `str = "JavaScript"`
- **Output:** `"tpircSavaJ"`

- **Example 3:**

- **Input:** `str = ""`
- **Output:** `""`
- **Explanation:** An empty string reversed remains empty.

Constraints

- $0 \leq \text{str.length} \leq 10^5$
- `str` consists of printable ASCII characters.

Starter Code (Try it yourself first!)

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {string}
 */
function reverseString(str) {
    // TODO: Write your code here
}
```

► Click to Reveal Solutions (With & Without Methods)

Problem 2: Reverse Words in a Sentence (Easy)

Problem Description

Write a function `reverseWords(sentence)` that takes a sentence string and returns a string with the words in reverse order.

- A word is defined as a non-empty sequence of non-whitespace characters.
- Leading or trailing spaces should be removed.
- Multiple contiguous spaces between words should be reduced to a single space.

Examples

- **Example 1:**

- **Input:** `sentence = "Cloud Storage Platform"`

- **Output:** "Platform Storage Cloud"
- **Example 2:**
 - **Input:** sentence = " hello world "
 - **Output:** "world hello"
- **Example 3:**
 - **Input:** sentence = "SingleWord"
 - **Output:** "SingleWord"

⚙️ Constraints

- $0 \leq \text{sentence.length} \leq 10^5$
- sentence contains English letters, digits, and spaces.

🚀 Starter Code

```
JAVASCRIPT

/**
 * @param {string} sentence
 * @return {string}
 */
function reverseWords(sentence) {
    // TODO: Write your code here
}
```

► 💡 Click to Reveal Solutions (With & Without Methods)

Problem 3: Count Characters (Easy)

📄 Problem Description

Write a function `countChars(str)` that returns the total count of characters in a string `str`.

📁 Examples

- **Example 1:**
 - **Input:** str = "hello"
 - **Output:** 5
- **Example 2:**
 - **Input:** str = ""
 - **Output:** 0
- **Example 3:**
 - **Input:** str = "JavaScript"
 - **Output:** 10

⚙️ Constraints

- $0 \leq \text{str.length} \leq 10^6$

 Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {number}
 */
function countChars(str) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 4: Count Vowels (Easy)

 Problem Description

Write a function `countVowels(str)` that counts the total number of vowels (`'a'`, `'e'`, `'i'`, `'o'`, `'u'`, case-insensitive) in a string `str`.

 Examples

- **Example 1:**

- **Input:** `str = "JavaScript"`
- **Output:** `3`
- **Explanation:** Vowels are `'a'`, `'a'`, `'i'`.

- **Example 2:**

- **Input:** `str = "rhythm"`
- **Output:** `0`

- **Example 3:**

- **Input:** `str = "AEIOU"`
- **Output:** `5`

 Constraints

- $0 \leq \text{str.length} \leq 10^5$

 Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {number}
 */
function countVowels(str) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 5: Count Consonants (Easy)

Problem Description

Write a function `countConsonants(str)` that counts the number of consonant letters in `str`. Consonants are alphabetic letters that are not vowels. Digits, spaces, and punctuation must be ignored.

Examples

- **Example 1:**

- **Input:** `str = "JavaScript"`
- **Output:** `7`

- **Example 2:**

- **Input:** `str = "Hello World! 123"`
- **Output:** `7`

Constraints

- Case-insensitive matching.

Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {number}
 */
function countConsonants(str) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 6: Count Words in a String (Easy)

Problem Description

Write a function `countWords(str)` that counts the total number of words in a string `str`.

- Contiguous whitespace (spaces, tabs, newlines) acts as a single boundary.
- If `str` contains only spaces or is empty, return `0`.

Examples

- **Example 1:**

- **Input:** `str = " High performance S3 storage "`
- **Output:** `4`

- **Example 2:**

- **Input:** `str = " "`
- **Output:** `0`

- **Example 3:**

- **Input:** `str = "SingleWord"`
- **Output:** `1`

⚙️ Constraints

- $\$0 \leq \text{text}\{\text{str.length}\} \leq 10^5$

🔗 Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {number}
 */
function countWords(str) {
  // TODO: Write your code here
}
```

▶ 💡 Click to Reveal Solutions (With & Without Methods)

Problem 7: Find String Length Without .length Property (Easy)

📄 Problem Description

Write a function `getLengthWithoutLength(str)` that returns the total count of characters in a string `str` **without accessing the `.length` property on any string or array.**

📁 Examples

• Example 1:

- **Input:** `str = "developer"`
- **Output:** `9`

• Example 2:

- **Input:** `str = ""`
- **Output:** `0`

⚙️ Constraints

- You **cannot** use the `.length` property anywhere in the solution.

🔗 Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {number}
 */
function getLengthWithoutLength(str) {
  // TODO: Write your code here without .length
}
```

▶ 💡 Click to Reveal Solutions (With & Without Methods)

Problem 8: Find First Character (Easy)

Problem Description

Write a function `getFirstChar(str)` that returns the first character of `str`.

- If `str` is empty (`""`), return `""` (do NOT return `undefined`).

Examples

- **Example 1:**

- **Input:** `str = "Engine"`
- **Output:** `"E"`

- **Example 2:**

- **Input:** `str = ""`
- **Output:** `""`

Constraints

- $0 \leq \text{str.length} \leq 10^5$

Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {string}
 */
function getFirstChar(str) {
    // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 9: Find Last Character (Easy)

Problem Description

Write a function `getLastChar(str)` that returns the final character of a string `str`. If `str` is empty, return `""`.

Examples

- **Example 1:**

- **Input:** `str = "NodeJS"`
- **Output:** `"S"`

- **Example 2:**

- **Input:** `str = "A"`
- **Output:** `"A"`

- **Example 3:**

- **Input:** `str = ""`
- **Output:** `""`

⚙️ Constraints

- $0 \leq \text{str.length} \leq 10^5$

🚀 Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {string}
 */
function getLastChar(str) {
  // TODO: Write your code here
}
```

▶ 💡 Click to Reveal Solutions (With & Without Methods)

Problem 10: Check Palindrome (Easy)

📄 Problem Description

Write a function `isPalindrome(str)` that determines if a string reads identically forwards and backwards.

📖 Examples

• Example 1:

- **Input:** `str = "racecar"`
- **Output:** `true`

• Example 2:

- **Input:** `str = "cloud"`
- **Output:** `false`

⚙️ Constraints

- $0 \leq \text{str.length} \leq 10^5$

🚀 Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {boolean}
 */
function isPalindrome(str) {
  // TODO: Write your code here
}
```

▶ 💡 Click to Reveal Solutions (With & Without Methods)

Problem 11: Remove All Spaces (Easy)

Problem Description

Write a function `removeSpaces(str)` that removes all whitespace characters from `str` and returns the contiguous result.

Examples

- **Example 1:**

- **Input:** `str = " Distributed Cloud Architecture "`
- **Output:** `"DistributedCloudArchitecture"`

- **Example 2:**

- **Input:** `str = "NoSpaces"`
- **Output:** `"NoSpaces"`

Constraints

- $0 \leq \text{str.length} \leq 10^5$

Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {string}
 */
function removeSpaces(str) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 12: Remove a Specific Character (Easy)

Problem Description

Write a function `removeChar(str, charToRemove)` that returns a new string with all occurrences of `charToRemove` eliminated.

Examples

- **Example 1:**

- **Input:** `str = "database", charToRemove = "a"`
- **Output:** `"dtbse"`

- **Example 2:**

- **Input:** `str = "hello", charToRemove = "z"`
- **Output:** `"hello"`

Constraints

- Case-sensitive removal.

 Starter Code

JAVASCRIPT

```
/**
 * @param {string} str
 * @param {string} charToRemove
 * @return {string}
 */
function removeChar(str, charToRemove) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 13: Replace a Character (Easy)

 Problem Description

Write a function `replaceChar(str, target, replacement)` that replaces every occurrence of `target` with `replacement`.

 Examples

- **Example 1:**

- **Input:** `str = "user-profile-data", target = "-", replacement = "_"`
- **Output:** `"user_profile_data"`

- **Example 2:**

- **Input:** `str = "2024/09/19", target = "/", replacement = "-"`
- **Output:** `"2024-09-19"`

 Constraints

- Target and replacement are strings.

 Starter Code

JAVASCRIPT

```
/**
 * @param {string} str
 * @param {string} target
 * @param {string} replacement
 * @return {string}
 */
function replaceChar(str, target, replacement) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 14: Count Occurrences of a Character (Easy)

Problem Description

Write a function `countChar(str, char)` that counts how many times a character `char` appears in `str`.

Examples

- **Example 1:**

- **Input:** `str = "kubernetes", char = "e"`
- **Output:** `3`

- **Example 2:**

- **Input:** `str = "aws", char = "z"`
- **Output:** `0`

Constraints

- Case-sensitive comparison.

Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @param {string} char
 * @return {number}
 */
function countChar(str, char) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 15: Find Character Frequency Map (Easy)

Problem Description

Write a function `charFrequency(str)` that returns an object mapping each character in `str` to its frequency count.

Examples

- **Example 1:**

- **Input:** `str = "storage"`
- **Output:** `{ s: 1, t: 1, o: 1, r: 1, a: 1, g: 1, e: 1 }`

- **Example 2:**

- **Input:** `str = "banana"`
- **Output:** `{ b: 1, a: 3, n: 2 }`

Constraints

- $0 \leq \text{str.length} \leq 10^5$

 Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {Record<string, number>}
 */
function charFrequency(str) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 16: Find Duplicate Characters (Easy)

 Problem Description

Write a function `findDuplicates(str)` that returns an array containing all characters that appear more than once in `str`.

- Each duplicate character must appear only once in the returned array.

 Examples

- **Example 1:**

- **Input:** `str = "programming"`
- **Output:** `["r", "g", "m"]`

- **Example 2:**

- **Input:** `str = "abcdef"`
- **Output:** `[]`

 Constraints

- Case-sensitive matching.

 Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {string[]}
 */
function findDuplicates(str) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 17: Remove Duplicate Characters (Easy)

 Problem Description

Write a function `removeDuplicates(str)` that removes all duplicate characters from `str`, preserving only their first occurrence.

 Examples• **Example 1:**

- **Input:** `str = "banana"`
- **Output:** `"ban"`

• **Example 2:**

- **Input:** `str = "google"`
- **Output:** `"gole"`

 Constraints

- Order of first appearance must be strictly preserved.

 Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {string}
 */
function removeDuplicates(str) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 18: Find First Non-Repeating Character (Easy / Medium)

 Problem Description

Write a function `firstNonRepeatingChar(str)` that returns the first character in `str` that does not repeat anywhere in the string. If none exists, return `null`.

 Examples• **Example 1:**

- **Input:** `str = "swiss"`
- **Output:** `"w"`

• **Example 2:**

- **Input:** `str = "aabbcc"`
- **Output:** `null`

• **Example 3:**

- **Input:** `str = "leetcode"`
- **Output:** `"l"`

 Constraints

- $0 \leq \text{str.length} \leq 10^5$

 Starter Code

JAVASCRIPT

```
/**
 * @param {string} str
 * @return {string | null}
 */
function firstNonRepeatingChar(str) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 19: Find First Repeating Character (Easy)

 Problem Description

Write a function `firstRepeatingChar(str)` that returns the first character that occurs for a second time as you read the string from left to right. If no characters repeat, return `null`.

 Examples

- **Example 1:**

- **Input:** `str = "google"`
- **Output:** `"o"`
- **Explanation:** Character `'o'` is seen at index 1 and repeats at index 2.

- **Example 2:**

- **Input:** `str = "abcdef"`
- **Output:** `null`

 Constraints

- $0 \leq \text{str.length} \leq 10^5$

 Starter Code

JAVASCRIPT

```
/**
 * @param {string} str
 * @return {string | null}
 */
function firstRepeatingChar(str) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 20: Convert First Character to Uppercase (Easy)

Problem Description

Write a function `capitalizeFirst(str)` that returns a new string where only the first character is capitalized, leaving all subsequent characters unchanged. If empty, return `""`.

Examples

- **Example 1:**

- **Input:** `str = "javascript"`
- **Output:** `"Javascript"`

- **Example 2:**

- **Input:** `str = "apple"`
- **Output:** `"Apple"`

- **Example 3:**

- **Input:** `str = ""`
- **Output:** `""`

Constraints

- $0 \leq \text{str.length} \leq 10^5$

Starter Code

```
JAVASCRIPT

/**
 * @param {string} str
 * @return {string}
 */
function capitalizeFirst(str) {
    // TODO: Write your code here
}
```

►  **Click to Reveal Solutions (With & Without Methods)**

Problem 21: Capitalize Every Word / Title Case (Easy)

Problem Description

Write a function `titleCase(sentence)` that capitalizes the first letter of each word and converts all other letters to lowercase.

Examples

- **Example 1:**

- **Input:** `sentence = "cloud storage system"`
- **Output:** `"Cloud Storage System"`

- **Example 2:**

- **Input:** `sentence = "jAvAsCrIpT iS aWeSoMe"`
- **Output:** `"Javascript Is Awesome"`

Constraints

- Words are separated by single spaces.

 Starter Code

```
JAVASCRIPT

/**
 * @param {string} sentence
 * @return {string}
 */
function titleCase(sentence) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 22: Find Longest Word in a Sentence (Easy)

Problem Description

Write a function `findLongestWord(sentence)` that returns the longest word in a sentence. Attached punctuation marks must not count toward word length. If multiple words tie for longest, return the first one.

Examples

- **Example 1:**

- **Input:** `sentence = "Build scalable distributed systems"`
- **Output:** `"distributed"` (11 letters)

- **Example 2:**

- **Input:** `sentence = "Fast, secure, reliable!"`
- **Output:** `"reliable"`

Constraints

- $0 \leq \text{sentence.length} \leq 10^5$

Starter Code

```
JAVASCRIPT

/**
 * @param {string} sentence
 * @return {string}
 */
function findLongestWord(sentence) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

Problem 23: Find Shortest Word in a Sentence (Easy)

Problem Description

Write a function `findShortestWord(sentence)` that returns the shortest word in a sentence. Ignore attached punctuation. If no words exist, return `""`.

 Examples• **Example 1:**

- **Input:** `sentence = "Deploy on AWS Cloud"`
- **Output:** `"on"` (length 2)

• **Example 2:**

- **Input:** `sentence = "A quick brown fox"`
- **Output:** `"A"` (length 1)

• **Example 3:**

- **Input:** `sentence = ""`
- **Output:** `""`

 Constraints

- $0 \leq \text{sentence.length} \leq 10^5$

 Starter Code

```
JAVASCRIPT

/**
 * @param {string} sentence
 * @return {string}
 */
function findShortestWord(sentence) {
  // TODO: Write your code here
}
```

►  Click to Reveal Solutions (With & Without Methods)

27. String Algorithms — Intermediate (24 to 49)

24. Check if Two Strings are Anagrams

- **Thought Process:** Two strings are anagrams if they contain the exact same character frequencies.
- **Code:**

```
JAVASCRIPT

function isAnagram(s, t) {
  if (s.length !== t.length) return false;
  const count = {};
  for (const c of s) count[c] = (count[c] ?? 0) + 1;
  for (const c of t) {
    if (!count[c]) return false;
    count[c]--;
  }
  return true;
}
console.log(isAnagram("listen", "silent")); // true
```

25. Group Anagrams

- **Code:**

JAVASCRIPT

```
function groupAnagrams(words) {
  const map = new Map();
  for (const w of words) {
    const key = w.split('').sort().join('');
    if (!map.has(key)) map.set(key, []);
    map.get(key).push(w);
  }
  return [...map.values()];
}
console.log(groupAnagrams(["eat", "tea", "tan", "ate", "nat", "bat"]));
// [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]
```

26. Character Frequency Comparison

- **Code:**

JAVASCRIPT

```
function areFrequenciesEqual(s1, s2) {
  const f1 = {};
  for (const c of s1) f1[c] = (f1[c] ?? 0) + 1;
  for (const c of s2) {
    if (!f1[c]) return false;
    f1[c]--;
  }
  return Object.values(f1).every(v => v === 0);
}
```

27. Check String Rotation

- **Thought Process:** If `s2` is a rotation of `s1`, it must be a substring of `s1 + s1`.
- **Code:**

JAVASCRIPT

```
function isRotation(s1, s2) {
  if (s1.length !== s2.length || s1.length === 0) return false;
  return (s1 + s1).includes(s2);
}
console.log(isRotation("waterbottle", "erbottlewat")); // true
```

28. Check Subsequence

- **Code:**

JAVASCRIPT

```
function isSubsequence(sub, full) {
  let i = 0, j = 0;
  while (i < sub.length && j < full.length) {
    if (sub[i] === full[j]) i++;
    j++;
  }
  return i === sub.length;
}
console.log(isSubsequence("ace", "abcde")); // true
```

29. Find All Duplicate Characters

- Code:

JAVASCRIPT

```
function getAllDuplicates(str) {
  const freq = {};
  for (const c of str) freq[c] = (freq[c] ?? 0) + 1;
  return Object.keys(freq).filter(c => freq[c] > 1);
}
```

30. Common Characters Between Two Strings

- Code:

JAVASCRIPT

```
function commonCharacters(s1, s2) {
  const set1 = new Set(s1);
  return [...new Set(s2)].filter(c => set1.has(c));
}
console.log(commonCharacters("apple", "pear")); // ["a", "p", "e"]
```

31. Remove Duplicate Words

- Code:

JAVASCRIPT

```
const removeDuplicateWords = (s) => [...new Set(s.split(/\s+/))].join(" ");
```

32. Count Word Frequency

- Code:

JAVASCRIPT

```
function wordFrequency(text) {  
  return text.toLowerCase().match(/\b\w+\b/g)?.reduce((acc, word) => {  
    acc[word] = (acc[word] ?? 0) + 1;  
    return acc;  
  }, {}) ?? {};  
}
```

33. Find Most Frequent Character

- **Code:**

JAVASCRIPT

```
function mostFrequentChar(str) {  
  const freq = {};  
  let maxChar = "", maxCount = 0;  
  for (const c of str) {  
    freq[c] = (freq[c] ?? 0) + 1;  
    if (freq[c] > maxCount) {  
      maxCount = freq[c];  
      maxChar = c;  
    }  
  }  
  return maxChar;  
}
```

34. Find Most Frequent Word

- **Code:**

JAVASCRIPT

```
function mostFrequentWord(str) {  
  const freq = wordFrequency(str);  
  let topWord = "", max = 0;  
  for (const [w, count] of Object.entries(freq)) {  
    if (count > max) { max = count; topWord = w; }  
  }  
  return topWord;  
}
```

35. String Compression (Run-Length Encoding)

- **Code:**

JAVASCRIPT

```
function compressString(str) {
  let compressed = "";
  let count = 1;
  for (let i = 0; i < str.length; i++) {
    if (str[i] === str[i + 1]) {
      count++;
    } else {
      compressed += str[i] + count;
      count = 1;
    }
  }
  return compressed.length < str.length ? compressed : str;
}
console.log(compressString("aabcccccaaa")); // "a2b1c5a3"
```

36. Run-Length Encoding Format

- **Code:**

JAVASCRIPT

```
function rleEncode(str) {
  return str.replace(/(.)\1*/g, (match, char) => `${char}${match.length}`);
}
```

37. Decode Compressed String

- **Code:**

JAVASCRIPT

```
function decodeString(compressed) {
  return compressed.replace(/([a-zA-Z])(\d+)/g, (_, char, count) => char.repeat(Number(count)));
}
console.log(decodeString("a2b1c5")); // "aabccccc"
```

38. Reverse Only Vowels

- **Code:**

JAVASCRIPT

```
function reverseVowels(str) {
  const vowels = new Set("aeiouAEIOU");
  const arr = str.split("");
  let left = 0, right = arr.length - 1;

  while (left < right) {
    while (left < right && !vowels.has(arr[left])) left++;
    while (left < right && !vowels.has(arr[right])) right--;
    if (left < right) {
      [arr[left], arr[right]] = [arr[right], arr[left]];
      left++;
      right--;
    }
  }
  return arr.join("");
}
console.log(reverseVowels("hello")); // "holLe"
```

39. Reverse Only Letters

- **Code:**

JAVASCRIPT

```
function reverseOnlyLetters(s) {
  const arr = s.split("");
  let left = 0, right = arr.length - 1;
  const isLetter = (c) => /[a-zA-Z]/.test(c);

  while (left < right) {
    if (!isLetter(arr[left])) left++;
    else if (!isLetter(arr[right])) right--;
    else {
      [arr[left], arr[right]] = [arr[right], arr[left]];
      left++;
      right--;
    }
  }
  return arr.join("");
}
console.log(reverseOnlyLetters("a-bC-dEf=ghIj!!")); // "j-Ih-gfE=dCb!a!"
```

40. Reverse Words While Preserving Spaces

- **Code:**

JAVASCRIPT

```
function reverseWordsPreserveSpaces(str) {
  const words = str.match(/\S+/g) ?? [];
  return str.replace(/\S+/g, () => words.pop());
}
console.log(reverseWordsPreserveSpaces(" sky is blue ")); // " blue is sky "
```

41. Move Spaces to Beginning

- **Code:**

JAVASCRIPT

```
function moveSpacesToFront(str) {  
  let spaces = "", nonSpaces = "";  
  for (const c of str) {  
    if (c === " ") spaces += " ";  
    else nonSpaces += c;  
  }  
  return spaces + nonSpaces;  
}
```

42. Valid Palindrome Ignoring Case & Spaces

- **Code:**

JAVASCRIPT

```
function isPalindromeAlphaNumeric(s) {  
  const clean = s.toLowerCase().replace(/^[^a-z0-9]/g, "");  
  return clean === clean.split("").reverse().join("");  
}  
console.log(isPalindromeAlphaNumeric("A man, a plan, a canal: Panama")); // true
```

43. Palindrome Ignoring Punctuation

- **Code:**

JAVASCRIPT

```
const isPalindromicSentence = (s) => isPalindromeAlphaNumeric(s);
```

44. Find Substring Occurrence Count

- **Code:**

JAVASCRIPT

```
function countSubstrings(main, sub) {  
  if (!sub) return 0;  
  return main.split(sub).length - 1;  
}  
console.log(countSubstrings("banana", "an")); // 2
```

45. Find All Indexes of a Substring

- **Code:**

JAVASCRIPT

```
function allIndexesOf(str, target) {
  const indices = [];
  let pos = str.indexOf(target);
  while (pos !== -1) {
    indices.push(pos);
    pos = str.indexOf(target, pos + target.length);
  }
  return indices;
}
console.log(allIndexesOf("banana", "an")); // [1, 3]
```

46. Compare Two Strings by Character Frequency

- Code:

JAVASCRIPT

```
function compareByFrequency(s1, s2) {
  const maxFreq = (s) => Math.max(...Object.values(charFrequency(s)));
  return maxFreq(s1) === maxFreq(s2);
}
```

47. Add Binary Strings

- Code:

JAVASCRIPT

```
function addBinary(a, b) {
  let i = a.length - 1, j = b.length - 1, carry = 0, result = "";
  while (i >= 0 || j >= 0 || carry) {
    const sum = (Number(a[i--] ?? 0)) + (Number(b[j--] ?? 0)) + carry;
    result = (sum % 2) + result;
    carry = Math.floor(sum / 2);
  }
  return result;
}
console.log(addBinary("11", "1")); // "100"
```

48. Add Very Large Numbers Represented as Strings

- Code:

JAVASCRIPT

```
function addLargeNumbers(num1, num2) {
  let i = num1.length - 1, j = num2.length - 1, carry = 0, result = [];
  while (i >= 0 || j >= 0 || carry) {
    const d1 = i >= 0 ? Number(num1[i--]) : 0;
    const d2 = j >= 0 ? Number(num2[j--]) : 0;
    const total = d1 + d2 + carry;
    result.push(total % 10);
    carry = Math.floor(total / 10);
  }
  return result.reverse().join("");
}
console.log(addLargeNumbers("9999999999999999999", "1")); // "10000000000000000000"
```

49. Multiply Large Numbers Represented as Strings

- **Code:**

JAVASCRIPT

```
function multiplyStrings(num1, num2) {
  if (num1 === "0" || num2 === "0") return "0";
  const m = num1.length, n = num2.length;
  const pos = new Array(m + n).fill(0);

  for (let i = m - 1; i >= 0; i--) {
    for (let j = n - 1; j >= 0; j--) {
      const mul = Number(num1[i]) * Number(num2[j]);
      const p1 = i + j, p2 = i + j + 1;
      const sum = mul + pos[p2];

      pos[p2] = sum % 10;
      pos[p1] += Math.floor(sum / 10);
    }
  }

  while (pos[0] === 0) pos.shift();
  return pos.join("");
}
console.log(multiplyStrings("123", "456")); // "56088"
```

28. String Algorithms — Advanced (50 to 67)

50. Longest Substring Without Repeating Characters (Sliding Window)

- **Algorithm:** Sliding window tracking last seen index of each character.
- **Code:**

JAVASCRIPT

```
function lengthOfLongestSubstring(s) {
  const lastIndex = new Map();
  let maxLen = 0, start = 0;

  for (let i = 0; i < s.length; i++) {
    const char = s[i];
    if (lastIndex.has(char) && lastIndex.get(char) >= start) {
      start = lastIndex.get(char) + 1;
    }
    lastIndex.set(char, i);
    maxLen = Math.max(maxLen, i - start + 1);
  }
  return maxLen;
}
console.log(lengthOfLongestSubstring("abcabcbb")); // 3 ("abc")
```

51. Longest Substring with At Most K Distinct Characters

- **Code:**

JAVASCRIPT

```
function lengthOfLongestSubstringKDistinct(s, k) {
  if (k === 0 || !s.length) return 0;
  const count = new Map();
  let left = 0, maxLen = 0;

  for (let right = 0; right < s.length; right++) {
    count.set(s[right], (count.get(s[right]) ?? 0) + 1);
    while (count.size > k) {
      count.set(s[left], count.get(s[left]) - 1);
      if (count.get(s[left]) === 0) count.delete(s[left]);
      left++;
    }
    maxLen = Math.max(maxLen, right - left + 1);
  }
  return maxLen;
}
console.log(lengthOfLongestSubstringKDistinct("eceba", 2)); // 3 ("ece")
```

52. Longest Substring with Exactly K Distinct Characters

- **Code:**

JAVASCRIPT

```
function longestSubstringExactK(s, k) {
  // At most K minus At most (K - 1)
  return lengthOfLongestSubstringKDistinct(s, k) - lengthOfLongestSubstringKDistinct(s, k - 1);
}
```

53. Minimum Window Substring

- **Code:**

JAVASCRIPT

```
function minWindow(s, t) {
  const targetMap = {};
  for (const c of t) targetMap[c] = (targetMap[c] ?? 0) + 1;

  let required = Object.keys(targetMap).length;
  let left = 0, right = 0, formed = 0;
  const windowCounts = {};
  let ans = [-1, 0, 0]; // length, left, right

  while (right < s.length) {
    const c = s[right];
    windowCounts[c] = (windowCounts[c] ?? 0) + 1;
    if (targetMap[c] && windowCounts[c] === targetMap[c]) formed++;

    while (left <= right && formed === required) {
      if (ans[0] === -1 || right - left + 1 < ans[0]) {
        ans = [right - left + 1, left, right];
      }
      const leftChar = s[left];
      windowCounts[leftChar]--;
      if (targetMap[leftChar] && windowCounts[leftChar] < targetMap[leftChar]) formed--;
      left++;
    }
    right++;
  }
  return ans[0] === -1 ? "" : s.slice(ans[1], ans[2] + 1);
}
console.log(minWindow("ADOBECODEBANC", "ABC")); // "BANC"
```

54. Find All Anagram Starting Positions

- **Code:**

JAVASCRIPT

```
function findAnagrams(s, p) {
  const result = [];
  if (s.length < p.length) return result;

  const pCount = Array(26).fill(0);
  const sCount = Array(26).fill(0);
  const code = (c) => c.charCodeAt(0) - 97;

  for (let i = 0; i < p.length; i++) {
    pCount[code(p[i])]++;
    sCount[code(s[i])]++;
  }

  if (pCount.join() === sCount.join()) result.push(0);

  for (let i = p.length; i < s.length; i++) {
    sCount[code(s[i])]++;
    sCount[code(s[i - p.length])]--;
    if (pCount.join() === sCount.join()) result.push(i - p.length + 1);
  }

  return result;
}
console.log(findAnagrams("cbaebabacd", "abc")); // [0, 6]
```

55. Longest Palindromic Substring

- **Thought Process:** Expand around center from every index (single character and pairs).
- **Code:**

JAVASCRIPT

```
function longestPalindrome(s) {
  if (!s || s.length < 2) return s;
  let start = 0, maxLen = 1;

  function expand(left, right) {
    while (left >= 0 && right < s.length && s[left] === s[right]) {
      const len = right - left + 1;
      if (len > maxLen) { start = left; maxLen = len; }
      left--;
      right++;
    }
  }

  for (let i = 0; i < s.length; i++) {
    expand(i, i); // Odd Length
    expand(i, i + 1); // Even Length
  }

  return s.slice(start, start + maxLen);
}
console.log(longestPalindrome("babad")); // "bab" (or "aba")
```

56. Count Palindromic Substrings

- **Code:**

JAVASCRIPT

```
function countSubstringsPalindromes(s) {
  let count = 0;
  function countExpand(left, right) {
    while (left >= 0 && right < s.length && s[left] === s[right]) {
      count++;
      left--;
      right++;
    }
  }
  for (let i = 0; i < s.length; i++) {
    countExpand(i, i);
    countExpand(i, i + 1);
  }
  return count;
}
console.log(countSubstringsPalindromes("aaa")); // 6 ("a", "a", "a", "aa", "aa", "aaa")
```

57. Longest Common Prefix

- **Code:**

JAVASCRIPT

```
function longestCommonPrefix(strs) {
  if (!strs.length) return "";
  let prefix = strs[0];
  for (let i = 1; i < strs.length; i++) {
    while (!strs[i].startsWith(prefix)) {
      prefix = prefix.slice(0, -1);
      if (!prefix) return "";
    }
  }
  return prefix;
}
console.log(longestCommonPrefix(["flower", "flow", "flight"])); // "fl"
```

58. Longest Common Suffix

- **Code:**

JAVASCRIPT

```
function longestCommonSuffix(strs) {
  if (!strs.length) return "";
  const reversed = strs.map(s => s.split("").reverse().join(""));
  return longestCommonPrefix(reversed).split("").reverse().join("");
}
```

59. Longest Common Substring (DP)

- **Code:**

JAVASCRIPT

```
function longestCommonSubstring(s1, s2) {
  const dp = Array.from({ length: s1.length + 1 }, () => Array(s2.length + 1).fill(0));
  let maxLen = 0, endIndex = 0;

  for (let i = 1; i <= s1.length; i++) {
    for (let j = 1; j <= s2.length; j++) {
      if (s1[i - 1] === s2[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1] + 1;
        if (dp[i][j] > maxLen) {
          maxLen = dp[i][j];
          endIndex = i;
        }
      }
    }
  }

  return s1.slice(endIndex - maxLen, endIndex);
}

console.log(longestCommonSubstring("storage_backup", "fast_storage_api")); // "storage_"
```

60. Longest Common Subsequence (LCS)

- **Code:**

JAVASCRIPT

```
function longestCommonSubsequence(text1, text2) {
  const m = text1.length, n = text2.length;
  const dp = Array.from({ length: m + 1 }, () => Array(n + 1).fill(0));

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (text1[i - 1] === text2[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1] + 1;
      } else {
        dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
      }
    }
  }

  return dp[m][n];
}

console.log(longestCommonSubsequence("abcde", "ace")); // 3 ("ace")
```

61. Edit Distance / Levenshtein Distance

- **Thought Process:** Minimum insertions, deletions, or substitutions to transform `word1` into `word2`.
- **Code:**

JAVASCRIPT

```
function minDistance(word1, word2) {
  const m = word1.length, n = word2.length;
  const dp = Array.from({ length: m + 1 }, () => Array(n + 1).fill(0));

  for (let i = 0; i <= m; i++) dp[i][0] = i;
  for (let j = 0; j <= n; j++) dp[0][j] = j;

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (word1[i - 1] === word2[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1];
      } else {
        dp[i][j] = 1 + Math.min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1]);
      }
    }
  }
  return dp[m][n];
}
console.log(minDistance("horse", "ros")); // 3
```

62. Word Break

- Code:

JAVASCRIPT

```
function wordBreak(s, wordDict) {
  const dict = new Set(wordDict);
  const dp = Array(s.length + 1).fill(false);
  dp[0] = true;

  for (let i = 1; i <= s.length; i++) {
    for (let j = 0; j < i; j++) {
      if (dp[j] && dict.has(s.slice(j, i))) {
        dp[i] = true;
        break;
      }
    }
  }
  return dp[s.length];
}
console.log(wordBreak("cloudstorage", ["cloud", "storage"])); // true
```

63. Decode Ways

- Code:

JAVASCRIPT

```
function numDecodings(s) {
  if (!s || s[0] === "0") return 0;
  let prev2 = 1, prev1 = 1;

  for (let i = 1; i < s.length; i++) {
    let current = 0;
    const oneDigit = Number(s[i]);
    const twoDigits = Number(s.slice(i - 1, i + 1));

    if (oneDigit >= 1) current += prev1;
    if (twoDigits >= 10 && twoDigits <= 26) current += prev2;

    prev2 = prev1;
    prev1 = current;
  }
  return prev1;
}
console.log(numDecodings("226")); // 3 ("BZ", "VF", "BBF")
```

64. String Interleaving

- **Code:**

JAVASCRIPT

```
function isInterleave(s1, s2, s3) {
  if (s1.length + s2.length !== s3.length) return false;
  const dp = Array.from({ length: s1.length + 1 }, () => Array(s2.length + 1).fill(false));
  dp[0][0] = true;

  for (let i = 0; i <= s1.length; i++) {
    for (let j = 0; j <= s2.length; j++) {
      if (i > 0) dp[i][j] = dp[i][j] || (dp[i - 1][j] && s1[i - 1] === s3[i + j - 1]);
      if (j > 0) dp[i][j] = dp[i][j] || (dp[i][j - 1] && s2[j - 1] === s3[i + j - 1]);
    }
  }
  return dp[s1.length][s2.length];
}
```

65. Isomorphic Strings

- **Code:**

JAVASCRIPT

```
function isIsomorphic(s, t) {
  const mapST = new Map(), mapTS = new Map();
  for (let i = 0; i < s.length; i++) {
    const c1 = s[i], c2 = t[i];
    if ((mapST.has(c1) && mapST.get(c1) !== c2) || (mapTS.has(c2) && mapTS.get(c2) !== c1)) {
      return false;
    }
    mapST.set(c1, c2);
    mapTS.set(c2, c1);
  }
  return true;
}
console.log(isIsomorphic("egg", "add")); // true
```

66. Pattern Matching

- **Code:**

JAVASCRIPT

```
function wordPattern(pattern, s) {
  const words = s.split(" ");
  if (pattern.length !== words.length) return false;
  const p2w = new Map(), w2p = new Map();

  for (let i = 0; i < pattern.length; i++) {
    const p = pattern[i], w = words[i];
    if ((p2w.has(p) && p2w.get(p) !== w) || (w2p.has(w) && w2p.get(w) !== p)) return false;
    p2w.set(p, w);
    w2p.set(w, p);
  }
  return true;
}
console.log(wordPattern("abba", "dog cat cat dog")); // true
```

67. Wildcard Matching ('?' and '*')

- **Code:**

JAVASCRIPT

```
function isWildcardMatch(s, p) {
  const m = s.length, n = p.length;
  const dp = Array.from({ length: m + 1 }, () => Array(n + 1).fill(false));
  dp[0][0] = true;

  for (let j = 1; j <= n; j++) {
    if (p[j - 1] === "*") dp[0][j] = dp[0][j - 1];
  }

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (p[j - 1] === s[i - 1] || p[j - 1] === "?") {
        dp[i][j] = dp[i - 1][j - 1];
      } else if (p[j - 1] === "*") {
        dp[i][j] = dp[i - 1][j] || dp[i][j - 1];
      }
    }
  }
  return dp[m][n];
}
console.log(isWildcardMatch("aa", "*")); // true
```

29. String Search Algorithms (68 to 73)

68. Naive Substring Search ($O(n \cdot m)$)

JAVASCRIPT

```
function naiveSearch(text, pattern) {
  const indices = [];
  for (let i = 0; i <= text.length - pattern.length; i++) {
    let match = true;
    for (let j = 0; j < pattern.length; j++) {
      if (text[i + j] !== pattern[j]) { match = false; break; }
    }
    if (match) indices.push(i);
  }
  return indices;
}
```

69 & 70. KMP (Knuth-Morris-Pratt) Algorithm & LPS Array ($O(n + m)$)

- **Why KMP exists:** Avoids re-checking characters that have already been matched by pre-computing the **Longest Proper Prefix which is also a Suffix (LPS)** array.

JAVASCRIPT

```
function buildLPS(pattern) {
  const lps = Array(pattern.length).fill(0);
  let len = 0, i = 1;
  while (i < pattern.length) {
    if (pattern[i] === pattern[len]) {
      len++;
      lps[i] = len;
      i++;
    } else {
      if (len !== 0) len = lps[len - 1];
      else { lps[i] = 0; i++; }
    }
  }
  return lps;
}

function kmpSearch(text, pattern) {
  const lps = buildLPS(pattern);
  const matches = [];
  let i = 0, j = 0;

  while (i < text.length) {
    if (text[i] === pattern[j]) { i++; j++; }
    if (j === pattern.length) {
      matches.push(i - j);
      j = lps[j - 1];
    } else if (i < text.length && text[i] !== pattern[j]) {
      if (j !== 0) j = lps[j - 1];
      else i++;
    }
  }
  return matches;
}

console.log(kmpSearch("abxabcabcaby", "abcaby")); // [6]
```

71 & 72. Rabin-Karp Algorithm & Rolling Hash

- **Concept:** Uses a rolling hash function to compare pattern hash with a sliding window hash of text in $O(1)$ time per shift.

JAVASCRIPT

```
function rabinKarp(text, pattern) {
  const d = 256, q = 101; // Base and prime modulus
  const m = pattern.length, n = text.length;
  let p = 0, t = 0, h = 1;
  const matches = [];

  for (let i = 0; i < m - 1; i++) h = (h * d) % q;
  for (let i = 0; i < m; i++) {
    p = (d * p + pattern.charCodeAt(i)) % q;
    t = (d * t + text.charCodeAt(i)) % q;
  }

  for (let i = 0; i <= n - m; i++) {
    if (p === t && text.slice(i, i + m) === pattern) matches.push(i);
    if (i < n - m) {
      t = (d * (t - text.charCodeAt(i) * h) + text.charCodeAt(i + m)) % q;
      if (t < 0) t = t + q;
    }
  }
  return matches;
}
console.log(rabinKarp("AABAACAADAABAABA", "AABA")); // [0, 9, 12]
```

73. Z-Algorithm

- Computes array `Z` where `Z[i]` is the length of the longest substring starting from `i` that matches the prefix of the string. Enables $O(n)$ pattern matching.

30. String Data Structures (74 to 79)

74 to 77. The Trie (Prefix Tree) Implementation

JAVASCRIPT

```
class TrieNode {
  constructor() {
    this.children = {};
    this.isEndOfWord = false;
  }
}

class Trie {
  constructor() {
    this.root = new TrieNode();
  }

  // 74. Insert Word
  insert(word) {
    let node = this.root;
    for (const char of word) {
      node.children[char] ??= new TrieNode();
      node = node.children[char];
    }
    this.isEndOfWord = true;
  }

  // 75. Search Full Word
  search(word) {
    let node = this.root;
    for (const char of word) {
      if (!node.children[char]) return false;
      node = node.children[char];
    }
    return node.isEndOfWord;
  }

  // 76. Prefix Search (startsWith)
  startsWith(prefix) {
    let node = this.root;
    for (const char of prefix) {
      if (!node.children[char]) return false;
      node = node.children[char];
    }
    return true;
  }

  // 77. Autocomplete suggestions
  autocomplete(prefix) {
    let node = this.root;
    for (const char of prefix) {
      if (!node.children[char]) return [];
      node = node.children[char];
    }
    const results = [];
    const dfs = (curr, currentWord) => {
      if (curr.isEndOfWord) results.push(currentWord);
      for (const [char, childNode] of Object.entries(curr.children)) {
        dfs(childNode, currentWord + char);
      }
    }
  }
}
```

```

    };
    dfs(node, prefix);
    return results;
  }
}

const trie = new Trie();
trie.insert("file");
trie.insert("filter");
trie.insert("filesystem");
console.log(trie.autocomplete("fil")); // ["file", "filter", "filesystem"]

```

78 & 79. Suffix Concepts & Suffix Array

A **Suffix Array** is a sorted array of all suffixes of a string. It allows $O(m \log n)$ pattern search using binary search over suffixes.

31. String Problem-Solving Patterns

TEXT

Pattern	When to Recognize	Template / Key Mechanic
Two Pointers	Palindromes, Reversals, Sorted	<code>left = 0; right = len - 1;</code>
Sliding Window	Substrings with constraints	<code>expand right; contract left;</code>
Frequency Counter	Anagrams, character counts	<code>map[c] = (map[c] ?? 0) + 1;</code>
Trie	Prefix queries, autocomplete	<code>node.children[char];</code>
Dynamic Programming	Edit distance, LCS, Word Break	<code>dp[i][j]</code> subproblem table

32. Real-World String Processing

1. File Path & Name Dissection

JAVASCRIPT

```

function dissectFilePath(fullPath) {
  const normalized = fullPath.replaceAll("\\", "/");
  const lastSlash = normalized.lastIndexOf("/");
  const dir = lastSlash !== -1 ? normalized.slice(0, lastSlash) : "";
  const file = lastSlash !== -1 ? normalized.slice(lastSlash + 1) : normalized;
  const lastDot = file.lastIndexOf(".");

  return {
    directory: dir,
    fullName: file,
    baseName: lastDot > 0 ? file.slice(0, lastDot) : file,
    extension: lastDot > 0 ? file.slice(lastDot + 1).toLowerCase() : ""
  };
}

console.log(dissectFilePath("C:\\users\\ayush\\documents\\report.2026.pdf"));

```

2. Robust CSV Line Parser (Handling Quoted Commas)

JAVASCRIPT

```
function parseCsvLine(line) {
  const result = [];
  let current = "";
  let inQuotes = false;

  for (let i = 0; i < line.length; i++) {
    const char = line[i];
    if (char === '"') {
      inQuotes = !inQuotes;
    } else if (char === ',' && !inQuotes) {
      result.push(current);
      current = "";
    } else {
      current += char;
    }
  }
  result.push(current);
  return result;
}

console.log(parseCsvLine('101,"Annual, Report",1048576'));
// ["101", "Annual, Report", "1048576"]
```

3. Log Line Parsing

JAVASCRIPT

```
function parseLogLine(log) {
  const regex = /^\[(<timestamp>[^\]]+\)\] \[(?<level>[A-Z]+\)\] (?<message>.+)$/;
  const match = log.match(regex);
  return match ? match.groups : null;
}

console.log(parseLogLine("[2026-09-12T11:00:00Z] [ERROR] Upload chunk 4 timeout"));
```

4. Text Highlighting

JAVASCRIPT

```
function highlightKeywords(text, keyword) {
  if (!keyword) return text;
  const escaped = keyword.replace(/[\.*+?^$()|[\]\\\]/g, "\\$&");
  const regex = new RegExp(`(${escaped})`, "gi");
  return text.replace(regex, "<mark>$1</mark>");
}

console.log(highlightKeywords("Cloud storage platform storage", "storage"));
// "Cloud <mark>storage</mark> pLatform <mark>storage</mark>"
```

33. Security: Defensive String Engineering

1. Cross-Site Scripting (XSS)

Never insert raw user strings into HTML. Always escape HTML special characters:

JAVASCRIPT

```
function sanitizeHtml(str) {  
  return String(str)  
    .replaceAll("&", "&")  
    .replaceAll("<", "<")  
    .replaceAll(">", ">")  
    .replaceAll("'", "'")  
    .replaceAll('"', '"');  
}
```

2. SQL Injection: Parameterization vs Naive Escaping

JAVASCRIPT

```
// ❌ UNSAFE: Never concatenate SQL strings  
// db.query(`SELECT * FROM users WHERE name = '${name}'`);  
  
// ✅ SAFE: Use parameterized queries  
// db.query('SELECT * FROM users WHERE name = $1', [name]);
```

3. Path Traversal Defense

Prevent users from escaping upload directories using `../`:

JAVASCRIPT

```
function safeJoin(baseDir, userInput) {  
  const safeName = path.basename(userInput); // Strips all directory components!  
  return path.join(baseDir, safeName);  
}
```

34. Performance: Immutability & Big Data

String Concatenation vs Array.join

In modern V8, concatenating a few strings with `+` is heavily optimized via `ConsString`. However, inside loops building large 10MB+ strings, pushing to an array and calling `.join("")` is faster and creates fewer intermediate GC allocations:

JAVASCRIPT

```
// Fast Loop concatenation:
const chunks = [];
for (let i = 0; i < 100000; i++) chunks.push("data_chunk_");
const fullPayload = chunks.join("");
```

35. JavaScript Engine / V8 Internals

 **Label:** The following are **V8 implementation details** (Chrome & Node.js). Other engines (SpiderMonkey in Firefox, JavaScriptCore in Safari) may use different internal architectures.

1. **SeqString:** Flat memory buffer for small static strings.
2. **ConsString (Rope Pattern):** Concatenations `a + b` create tree nodes referencing both parents until flattened.
3. **SlicedString:** Substrings create `[parentPointer, offset, length]`.
 - **The Memory Leak:** Slicing 10 characters from a 50MB buffer retains the entire 50MB parent in memory.
 - **Fix:** Force a new flat string allocation:

JAVASCRIPT

```
const detachedSlice = (" " + hugeStr.slice(0, 10)).slice(1);
```

36. Debugging & Common Footguns

TEXT

 **WRONG:** `str.trim();` (Result discarded! Strings are immutable)

 **RIGHT:** `str = str.trim();`

 **WRONG:** `str.replace("a", "b");` (Only replaces first occurrence!)

 **RIGHT:** `str.replaceAll("a", "b");`

 **WRONG:** `str.split(".");` (In regex, `.` matches everything!)

 **RIGHT:** `str.split(".");` (Pass literal string, not `new RegExp(".")`)

37. Testing & Edge Case Checklist

When testing any string function, verify these test vectors:

- Empty string `""`
- Single character `"a"`
- Repeated characters `"aaaa"`
- Whitespace only `" \t\n"`
- Unicode emojis `"👉"` and compound emojis `"👨👩👧"`

- Special regex metacharacters `"[.*+?^$]"`
- Extremely long string (10,000+ characters)

38. Interview Preparation

Q1: Why are strings immutable in JavaScript?

- **Answer:** Immutability allows strings to be safely shared across memory threads without locking, permits string interning (reusing identical string literals from a hash pool), and guarantees that object property names cannot change unexpectedly.

Q2: What is the difference between `slice()` and `substring()`?

- **Answer:** `slice` supports negative indices counting from the end and returns `""` if `start > end`. `substring` clamps negative numbers to `0` and automatically swaps arguments if `start > end`.

Q3: Why does `"📁".length` return 2?

- **Answer:** JavaScript `.length` counts 16-bit UTF-16 code units. The folder emoji has code point `U+1F4C1` which exceeds 16 bits and is represented by a 2-unit surrogate pair.

39. Practice System (Graded Exercises)

🟢 Easy: Clean and Format Filename

- Input: `" My Report.PDF "`
- Output: `"my_report.pdf"`

🟡 Medium: Find First Non-Repeating Character

- Input: `"swiss"`
- Output: `"w"`

🔴 Hard: Longest Substring Without Repeating Characters

- Input: `"abcabcbb"`
- Output: `3` (`"abc"`)

🟡 Senior: Safe Path & URL Tokenizer with Param Extraction

40. Final Projects

Project 1 (Beginner): Text Analyzer

Analyzes input text and reports: Total characters, total words, vowels, consonants, reading time.

Project 2 (Intermediate): Smart Markdown Slug & Heading Generator

Converts `#### 12. Complete Guide to UTF-16 & Emojis!` into a clean URL anchor slug: `"12-complete-guide-to-utf-16-emojis"`.

Project 3 (Advanced): Mini In-Memory Text Search Engine

Tokenizes a corpus of documents, indexes terms in a Map/Trie, and provides prefix matching with keyword highlighting.

Project 4 (Senior): Production URL & Query String Parsing Pipeline

Fully RFC-compliant URL parser without third-party dependencies, handling protocols, hostnames, ports, decoded parameters, and hash anchors.

41. String Method Cheat Sheet

TEXT

Method	Purpose	Returns	Mutates?
<code>str.slice(start, end)</code>	Extract substring	string	NO
<code>str.substring(start, end)</code>	Legacy extract (auto-swaps)	string	NO
<code>str.includes(search)</code>	Check existence	boolean	NO
<code>str.indexOf(search)</code>	Find first index	number (-1)	NO
<code>str.lastIndexOf(search)</code>	Find last index	number (-1)	NO
<code>str.startsWith(prefix)</code>	Check prefix	boolean	NO
<code>str.endsWith(suffix)</code>	Check suffix	boolean	NO
<code>str.replace(a, b)</code>	Replace first match	string	NO
<code>str.replaceAll(a, b)</code>	Replace all matches	string	NO
<code>str.split(separator)</code>	String -> Array	Array	NO
<code>str.trim()</code>	Strip whitespace edges	string	NO
<code>str.padStart(len, pad)</code>	Pad left side	string	NO
<code>str.padEnd(len, pad)</code>	Pad right side	string	NO
<code>str.toLowerCase()</code>	Lowercase	string	NO
<code>str.toUpperCase()</code>	Uppercase	string	NO
<code>str.localeCompare(other)</code>	Alphabetical sort comparison	number (-1,0,1)	NO
<code>str.normalize("NFC")</code>	Unicode canonical form	string	NO
<code>str.at(index)</code>	Access char with -index	string/undef	NO
<code>str.codePointAt(index)</code>	32-bit Unicode code point	number	NO
<code>String.fromCodePoint(code)</code>	Code point -> String	string	NO

42. Method Decision Tree ("What Do I Need to Do?")

TEXT

```

Need a single character?
├─ By index (positive or negative) → .at(index)
└─ First / Last → .at(0) / .at(-1)

Need a piece of the string?
└─ From index A to B → .slice(start, end)

Need to check contents?
├─ Exists anywhere? → .includes(str)
├─ Starts with? → .startsWith(str)
├─ Ends with? → .endsWith(str)
└─ Where is it? → .indexOf(str) / .lastIndexOf(str)

Need to change contents?
├─ First match only → .replace(str, newStr)
├─ All matches → .replaceAll(str, newStr)
├─ Clean edges → .trim()
└─ Format length → .padStart() / .padEnd()

Need to turn into a list? → .split(delimiter)

Need to count human characters? → Intl.Segmenter

```

43. Common Mistakes Catalog

TEXT

```

❌ const s = "hello"; s.toUpperCase(); console.log(s); // Still "hello"!
✅ const s = "hello"; const upper = s.toUpperCase();

❌ str.replace("a", "b"); // Expecting all "a"s to change
✅ str.replaceAll("a", "b");

❌ str.slice(str.length - 3);
✅ str.slice(-3);

❌ typeof new String("test") === "string" // FALSE! It's "object"
✅ typeof "test" === "string"

❌ "📁".length === 1 // FALSE! It's 2!
✅ [...new Intl.Segmenter().segment("📁")].length === 1

```

44. Final Senior Checklist

Before deploying string manipulation to production:

- ☐ Are all return values assigned or returned? (Strings are immutable!)
- ☐ Is input sanitized against path traversal (`../`)?

- ☐ Are database queries parameterized instead of string concatenated?
- ☐ Are regex patterns protected against catastrophic backtracking (ReDoS)?
- ☐ Are emojis and Unicode graphemes handled using `Intl.Segmenter` or `codePointAt` ?
- ☐ Are file extensions normalized with `.toLowerCase()` ?
- ☐ Are numeric string lists sorted with `localeCompare(..., { numeric: true })` ?
- ☐ Are edge cases tested: `""`, nullish inputs, and whitespace-only strings?